

A PRACTITIONER'S FIELD GUIDE

Detection Engineering in Practice

Building, running, and tuning a YARA-based SOC detector

Most security writing stops at the rule. This book follows a single real detector all the way through — from the first YARA string to the alert a tired analyst has to judge at 3 a.m. Every example is drawn from one working, published codebase, including the bugs found while building it, the vulnerability feeds bolted on afterwards, and the TLS failure that was not the server's fault.

Larry Allen

github.com/gocar112/security-suite-dashboard

Fourth edition — September 2026

Contents

FRONT MATTER	Preface	3
PART ONE	The Discipline	5
CHAPTER ONE	The Detection Engineer's Job	6
CHAPTER TWO	The File Drop as an Attack Surface	9
PART TWO	Writing Detections	12
CHAPTER THREE	YARA, Precisely	13
CHAPTER FOUR	Rules That Survive Contact with Reality	18
PART THREE	Building the Sensor	24
CHAPTER FIVE	Anatomy of a Detector	25
CHAPTER SIX	The Race Conditions Nobody Warns You About	32
CHAPTER SEVEN	Telemetry, Time, and Correlation	38
PART FOUR	Operating the Detector	46
CHAPTER EIGHT	Severity, Triage, and the Economics of Attention	47
CHAPTER NINE	Measuring and Tuning a Ruleset	51
CHAPTER TEN	Converging with Vulnerability Intelligence	56
CHAPTER ELEVEN	From Detection to Pivot	67
CHAPTER TWELVE	Running It for Real	72
APPENDICES	Reference	76
APPENDIX A	The Complete Ruleset	77
APPENDIX B	HTTP API Reference	85
APPENDIX C	Configuration Reference	88
APPENDIX D	Command Line and Recipes	91

Preface

This book has one detector in it. Not a survey of twelve tools, not a vendor comparison — one small, complete, working sensor, followed from the first line of a YARA rule to the moment an analyst decides whether an alert was worth their attention.

There is a lot of security writing that explains what a detection rule *is*. There is much less that explains what happens to that rule once it meets a real filesystem, a real antivirus product, a real log with missing timestamps, and a real person who has forty other alerts open. The gap between those two things is where detection engineering actually lives, and it is almost entirely made of unglamorous problems: files that are still being written when you scan them, logs you do not have permission to read, a severity field nobody agreed on, and an alert queue that grows faster than anyone can clear it.

The running example throughout is **Security Suite**, a YARA-backed file detector with a live dashboard, published at github.com/gocar112/security-suite-dashboard. It is small enough to read in an afternoon — roughly 1,400 lines of Python and 1,000 of front-end — and complete enough to have hit every class of problem this book describes. Where the text says "this broke", it broke. Where it says "the fix was", you can read the fix.

Who this is for

You should be comfortable reading Python and willing to look at a regular expression without flinching. You do not need prior YARA experience; Chapter 3 builds it from nothing. You do not need to have worked in a SOC, though if you have, several chapters will read as confirmation rather than instruction.

The book is aimed at three people in particular. The **analyst** who is tired of tuning someone else's rules and wants to understand the machinery underneath them. The **developer** who has been handed "add malware scanning to the upload endpoint" and correctly suspects it is harder than it sounds. And the **student or career-changer** who wants a portfolio project that demonstrates judgment rather than just syntax.

How to read it

Part I frames the discipline and the threat model. Part II is about writing detection content — YARA specifically, but the reasoning generalizes to Sigma, Suricata, and anything else with a condition block. Part III builds the sensor and is where most of the hard-won material lives: the race conditions, the timestamp bug, the feedback loop that made the detector alert on itself. Part IV is about operating the thing once it exists, which is the part most projects skip and most deployments fail on.

Chapters are meant to be read in order, but Part III and Part IV stand alone well enough if you already write rules for a living. The appendices are reference material: the full ruleset, the HTTP API, every configuration key, and the command line.

A NOTE ON THE CODE

Every code listing is real, taken from the published repository rather than written for the page. Line-level details may drift as the project evolves; the reasoning will not. Clone it, break it, and watch what the dashboard says.

On the word "practice"

The title is not decoration. A detection that works on your laptop against a file you crafted yourself is a demonstration. A detection that works on a machine you do not control, against a file someone else crafted, while an antivirus product is actively deleting your evidence, is a practice. This book is about the second kind, and it gets there by making the first kind fail in instructive ways.

PART ONE

The Discipline

What detection engineering is, what it is responsible for, and the specific corner of it this book lives in.

The Detection Engineer's Job

Detection engineering is the practice of turning knowledge about attacker behaviour into automated signal that a human can act on — and then taking responsibility for what that signal does to the human.

The second half of that sentence is the part that gets dropped. It is straightforward to write a rule that fires on something bad. It is considerably harder to write one that fires on something bad, does not fire on the nine thousand benign files that look vaguely similar, arrives with enough context that an analyst can judge it in under a minute, and keeps behaving that way six months later when the environment has changed and nobody remembers why the rule exists.

The detection lifecycle

Detection work runs in a loop, and every stage of it can fail independently:

1. **Hypothesis.** A belief about attacker behaviour. "An attacker who gets file-upload access will drop a web shell." This comes from threat intelligence, incident history, red team findings, or reasoning about your own architecture.
2. **Instrumentation.** Can you even see the thing your hypothesis is about? If files land in a directory no process watches, no rule will ever fire. Visibility precedes detection and is usually the real bottleneck.
3. **Detection content.** The rule itself — the YARA condition, the Sigma correlation, the query. This is the part everyone writes about.
4. **Enrichment.** The context attached to a hit: file hash, who uploaded it, what else happened around the same time. An alert without enrichment is a question, not an answer.
5. **Triage.** A human decides: real, not real, or needs investigation. Everything upstream exists to make this decision fast and correct.
6. **Tuning.** What you learn in triage feeds back into the rule. A detection that is never tuned is a detection that is decaying.

Security Suite implements all six, deliberately and visibly. The hypothesis is encoded in `rules/`. The instrumentation is the directory monitor. The content is YARA. The enrichment is the SHA-256, the entropy, the matched strings, and the correlated authentication failures. The triage is four buttons in a web UI. The tuning is a rule file you edit and hot-reload. Being able to point at each stage in a system small enough to hold in your head is the entire reason this book uses one codebase instead of many.

Three ways a detection fails

It helps to be precise about failure, because the three modes have different costs and pull in different directions.

The missed detection

The attacker does the thing, and nothing fires. This is the failure everyone fears and the hardest to measure, because by definition you do not have a record of it. You discover false negatives through incident response, red teaming, or purple team exercises — never through your alert queue, which is exactly the trap.

The false positive

Something benign fires the rule. Each one is individually cheap and collectively ruinous. A rule with a 2% false positive rate sounds excellent until you run it against 50,000 files a day and it generates a thousand alerts nobody will ever read.

The unactionable alert

This is the underrated one. The rule fired, it was even correct, and the analyst still cannot do anything with it — because it says `MALWARE_DETECTED` and nothing else. No file path, no hash, no indication of what matched or why. It is technically a true positive and operationally worthless.

Notice that these trade against each other. Broadening a rule to reduce false negatives increases false positives. Narrowing it does the reverse. Adding enrichment costs scan time and complexity. Every detection is a position on those axes, and a good detection engineer can state where their rule sits and why.

THE ASYMMETRY WORTH REMEMBERING

A false negative costs you one incident. A high false positive rate costs you every incident, because it trains the analyst to stop looking. Alert fatigue is not a morale problem; it is a detection failure with a delayed fuse.

Detection versus prevention

Security Suite detects. It does not quarantine, delete, block, or move anything. That is a deliberate architectural choice, and it is worth understanding why.

Prevention has a failure mode detection does not: when prevention is wrong, it breaks production. An antivirus product that false-positives on your deployment artifact does not generate an alert somebody can dismiss — it deletes your build and takes the service down. The blast radius of a wrong preventive

action is enormous, which is why preventive controls must be held to a far higher precision standard than detective ones.

Detection is allowed to be wrong. That freedom is what lets a detection engineer ship a rule at 80% precision, watch what it catches, and tune it — a process that would be unacceptable if every firing deleted a customer file. Keep them separate, and be suspicious of any tool that quietly blurs the line.

Where this book sits

Detection spans network, endpoint, identity, cloud control plane, and application layers. This book lives almost entirely in one narrow place: **files arriving on disk**. That narrowness is a feature. File-based detection is the oldest and best-understood corner of the discipline, it has a mature tool in YARA, and every hard problem in the wider field — visibility gaps, timestamp handling, severity taxonomy, alert economics — shows up there in miniature.

What you learn here about correlating a file event with an authentication event transfers directly to correlating a process launch with a DNS query. The plumbing changes; the reasoning does not.

The File Drop as an Attack Surface

Before writing a single rule, you need a threat model: who is putting files where, and what do they want those files to do?

A rule written without a threat model is a rule written against your imagination. It will catch what you happened to think of and miss what you did not, and you will have no way of knowing which is which. So this chapter establishes the specific threat model the case study defends, and the reasoning applies to whatever directory you end up watching.

Why directories are where this starts

Almost every system that accepts input from the outside world eventually writes that input to disk. Upload forms, FTP drop boxes, email attachment processors, document conversion pipelines, CI artifact stores, network shares, backup landing zones. Each one is a place where content controlled by someone else comes to rest inside your trust boundary.

That makes watched directories unusually high-value for detection. The attacker has to materialize something — a payload, a tool, a script — and for a moment it sits still, unencrypted, in a place you control. Network detection sees a stream you may not be able to decrypt. Process detection sees behaviour only after execution begins. File detection gets the artifact itself, at rest, before it runs.

The five things that land in an uploads folder

1. Web shells

The classic. An attacker who finds an upload form that does not validate file types drops a small script — PHP, ASPX, JSP — into a directory the web server will execute. From then on they have command execution, and it looks like ordinary web traffic.

Web shells are an excellent detection target because they have an irreducible structure. To be useful, a web shell must take input from the request and pass it to something that executes. It can be obfuscated in a thousand ways, but those two elements have to be present, and that pairing is what a good rule keys on. Chapter 4 develops exactly this.

2. Ransomware notes and staging

By the time a ransom note appears, encryption is usually well underway — so detecting the note is late. It is still worth doing, for two reasons: it is unambiguous (almost nothing benign says "your files have been encrypted" next to a payment address), and it gives you a precise timestamp for incident

reconstruction. Earlier signals — shadow copy deletion, security tooling being disabled — are more valuable, and the case study covers both.

3. Offensive tooling

Credential dumpers, lateral movement utilities, enumeration scripts. These arrive after initial access, when the attacker is expanding. They are often well-known public tools with well-known strings, which makes them easy to detect and correspondingly easy for a competent adversary to modify. Detecting the unmodified version is still worth doing: most intrusions are not run by competent adversaries.

4. Fileless payloads written to disk anyway

The industry says "fileless" and then writes an encoded PowerShell command into a scheduled task definition, a log file, a registry export, or a batch script. The payload does not persist as an executable, but the *command* frequently does persist as text somewhere on disk — which is why the case study's ruleset looks for encoded command patterns and download cradles in plain text files, not only in binaries.

5. Social engineering artifacts

Double extensions (`invoice.pdf.exe`), documents with macro lures, LNK files pointing at scripts. These target the user rather than the system, and they show up in mail stores and download folders more than in server upload directories.

The threat model, stated

Putting that together, here is what Security Suite is built to defend, written plainly enough to argue with:

Element	Assumption	Consequence for design
Adversary	Opportunistic to moderately skilled. Uses public tooling, may lightly obfuscate, is not writing a bespoke packer for this target.	String and pattern matching is worth doing. Heavy anti-evasion machinery is not the first priority.
Access	Can write files into a watched directory, by upload, transfer, or compromised account.	The directory is the detection point. Files are untrusted on arrival.
Goal	Execute code, persist, collect credentials, or extort.	Rules target execution primitives and extortion artifacts, not file types.
Defender	One or a few people, not a 24/7 tier-one team.	Alert volume must stay low. Every alert must be judgeable in under a minute.
Environment	Windows-first, Linux-capable. Antivirus is present and active.	Telemetry reads the Windows Security log. The scanner must survive AV deleting its evidence.

WHAT THIS MODEL EXCLUDES

A nation-state adversary writing a custom loader for your specific environment will not be caught by string matching, and pretending otherwise is how security programmes waste years. This detector raises the cost of the common case. That is a real and worthwhile goal — it is simply not the same goal as defeating a determined, well-resourced attacker, and conflating the two produces bad architecture and worse expectations.

The one question to ask before every rule

For each rule you write, you should be able to answer: **what does a true positive look like, and what will I do about it?**

If the answer to the first half is vague, the rule is a guess. If the answer to the second half is "nothing, really", the rule is noise you have not yet noticed. Detection content that nobody would act on should not exist; it dilutes attention away from content that matters. This is the single most effective filter for keeping a ruleset healthy, and the easiest to skip.

PART TWO

Writing Detections

YARA from first principles, and the difference between a rule that matches and a rule that holds up.

YARA, Precisely

YARA is a pattern matching language for classifying files. This chapter builds it from nothing, using only constructs the case study actually uses.

YARA has a reputation as "grep for malware", which undersells it badly. Grep asks whether a pattern appears. YARA asks whether a *combination* of patterns appears, in particular positions, in a file of a particular size, and it evaluates that question fast enough to run against every file on a filesystem.

Anatomy of a rule

Every YARA rule has the same three-part shape. Here is one from the case study, complete:

```
rule PowerShell_Encoded_Command : suspicious
{
    meta:
        description = "PowerShell launched with a base64 encoded command block"
        severity    = "high"
        author      = "security-suite"
    strings:
        $ps = "powershell" nocase
        $e1 = "-encodedcommand" nocase
        $e2 = " -enc " nocase
        $e3 = "-e JAB" nocase
        $e4 = "FromBase64String" nocase
    condition:
        $ps and any of ($e*)
}
```

Reading it top to bottom: `rule` names it, `: suspicious` attaches a tag, `meta:` holds arbitrary key-value pairs that do not affect matching, `strings:` declares patterns, and `condition:` is the boolean expression that decides. Only the condition determines a match. Metadata is for humans and for downstream tooling — which, as Chapter 8 shows, is how severity gets into the dashboard.

Strings

YARA has three kinds of string, and knowing when to reach for each is most of the craft.

Text strings

A literal sequence of characters, optionally decorated with modifiers:

```
$a = "sekurlsa::logonpasswords" nocase
$b = "MZ" ascii
$c = "Invoke-Expression" nocase wide
$d = "eval" fullword
```

Modifier	Effect
<code>nocase</code>	Case-insensitive. Use it by default for anything human-typed — command lines, script keywords, note text. Costs a little performance, prevents a large class of trivial evasion.
<code>wide</code>	Matches UTF-16LE encoding, where each character is followed by a null byte. Windows binaries store most strings this way. A rule looking for a Windows API name without <code>wide</code> will silently miss.
<code>ascii</code>	Matches single-byte encoding. This is the default, but becomes explicit and necessary the moment you add <code>wide</code> — <code>wide</code> alone replaces ASCII matching rather than adding to it.
<code>fullword</code>	Requires non-alphanumeric boundaries. <code>"eval" fullword</code> matches <code>eval(</code> but not <code>medieval</code> . Essential for short strings.
<code>xor</code>	Matches the string XOR-encoded against single-byte keys. Catches lazy obfuscation; expands the search space, so use deliberately.
<code>base64</code>	Matches the base64 encodings of the string. Useful against encoded configuration blobs.

THE WIDE/ASCII TRAP

This is the most common beginner mistake in YARA, and it fails silently. Writing `$a = "CreateRemoteThread"` against a Windows PE will often not match, because the string is stored as UTF-16. Writing `$a = "CreateRemoteThread" wide` will then miss any ASCII copy. What you almost always want is `wide ascii` — both. The rule compiles and runs either way, which is precisely why the mistake survives.

Hexadecimal strings

For binary content, expressed as bytes with wildcards:

```
$mz      = { 4D 5A }           // exact: the PE magic bytes "MZ"
$stub    = { 4D 5A ?? ?? 00 00 } // ?? = any single byte
$range   = { E8 [4-8] 5D C3 }   // [n-m] = between n and m arbitrary bytes
$alt     = { 6A 40 ( 68 | 6A ) } // ( | ) = alternatives
```

Hex strings are how you match file structure rather than file content: headers, magic bytes, instruction sequences, embedded structures. The case study uses exactly one, `{ 4D 5A }`, to identify PE files — and pairs it with a position anchor, which is the subject of the next section.

Regular expressions

YARA supports regex between forward slashes: `/https?:\\/[a-z0-9.-]+\\.onion/ nocase`. They are powerful and considerably slower than text or hex strings. The working rule is to reach for regex only when the alternatives genuinely cannot express the pattern — a handful of text strings combined in the condition usually can, and will run far faster.

Conditions

The condition is a boolean expression, and it is where a rule becomes precise rather than merely present.

Counting and sets

```
condition:
    $a                                // this string appears at least once
    any of them                       // at least one declared string
    all of them                       // every declared string
    2 of ($a, $b, $c)                // at least two of these three
    any of ($e*)                     // at least one string named $e-something
    #a > 3                            // $a appears more than three times
    #a == 1                           // exactly once
```

The `any of ($prefix*)` form is the workhorse. It lets you declare a family of related indicators and require one, which is how you express "any of these several ways of spelling the same idea" without a wall of `or`.

Positions and size

```
condition:
    $mz at 0                          // string occurs exactly at offset 0
    $sig in (0..1024)                 // occurs somewhere in the first KB
    @a[1] < @b[1]                     // first $a occurs before first $b
    filesize < 20MB                   // file size bound
    uint16(0) == 0x5A4D                // read two bytes at offset 0 as an integer
```

Position anchoring is a precision multiplier and an underused one. `$mz at 0` means "this file begins with MZ", which is meaningfully different from "the letters MZ appear somewhere in this file" — the latter matches any text document containing the word *amazing*. Free precision, one keyword.

Modules

YARA ships modules that expose computed properties. The case study imports one:

```

import "math"

rule High_Entropy_Executable : suspicious
{
    meta:
        description = "PE file whose body looks packed or encrypted"
        severity     = "medium"
    strings:
        $mz = { 4D 5A }
    condition:
        $mz at 0 and math.entropy(0, filesize) > 7.2 and filesize < 20MB
}

```

`math.entropy` computes Shannon entropy over a byte range. The `pe` module goes much further — sections, imports, exports, signatures, resources — and is worth learning once you are matching on binaries in earnest. Note that `import` must appear at the top of the file, before any rule; forgetting this is a compile error that reads more confusingly than it should.

Namespaces, tags, and organisation

When you compile several rule files together, each becomes a **namespace**. Rule names must be unique within a namespace, not globally, which means two teams can both have a rule called `Suspicious_Archive` without collision. The case study compiles one namespace per file:

```

sources = {}
for path in rule_files:
    namespace = path.stem
    sources[namespace] = path.read_text()

compiled = yara.compile(sources=sources)

```

Tags (the `: suspicious` after a rule name) are a second, orthogonal axis — a rule can carry several. They are returned with every match, which makes them useful for routing: send everything tagged `ransomware` to the on-call phone, everything tagged `test` to a log nobody pages on.

Compiling and matching in Python

The case study uses `yara-python` rather than shelling out to the `yara` binary. This matters more than it sounds: compiling rules once and holding the compiled object in memory removes both process spawn overhead and recompilation from every single scan.


```

import yara

rules = yara.compile(filepath="rules/webshell.yar")      # one file
rules = yara.compile(sources={"ns": source_text})      # from strings
rules = yara.compile(filepaths={"ns": "rules/a.yar"})  # several files

matches = rules.match(filepath="suspect.bin", timeout=60)
matches = rules.match(data=buffer)                    # in-memory bytes

for m in matches:
    print(m.rule, m.namespace, m.tags, m.meta)
    for s in m.strings:                                # StringMatch
        for i in s.instances:                          # StringMatchInstance
            print(s.identifier, hex(i.offset), i.matched_data)

```

API VERSION NOTE

The nested `strings / instances` structure above is the modern yara-python API (4.3 and later). Older code returns flat `(offset, identifier, data)` tuples, and a great deal of tutorial material still shows that form. If a match-iteration example fails with an attribute error, an API generation mismatch is the first thing to check. The case study targets 4.5.

The timeout argument nobody passes

`rules.match(filepath=..., timeout=60)` bounds how long matching may run. It is optional and routinely omitted, which is a mistake in any long-running scanner: a pathological file combined with an expensive regex can hang a scan indefinitely, and a monitor blocked forever on one file is a monitor that has silently stopped protecting you. Setting a timeout converts an invisible hang into a visible error.

Rules That Survive Contact with Reality

A rule that matches your test file is not a detection. It becomes one when it still behaves after meeting a hundred thousand files you did not choose.

This chapter is about the difference. It works through the case study's ruleset, rule by rule, and names the technique each one demonstrates.

The single-string trap

The obvious first web shell rule is the wrong one:

```
rule Bad_Webshell
{
    strings:
        $a = "eval(" nocase
    condition:
        $a
}
```

This fires on every JavaScript library that has ever shipped a polyfill, every minifier output, every piece of documentation about `eval`, and a large fraction of legitimate PHP. It is not a detection; it is a fuzzy `grep` that produces alerts. The rule is not *wrong* about `eval` being interesting — it is wrong about `eval` alone being sufficient.

Two-factor conditions

The fix is to require two independent elements that are individually common and jointly rare. Here is the case study's actual rule:

```

rule PHP_Webshell_Eval_Input : webshell
{
    meta:
        description = "PHP one-liner passing request data straight to an evaluator"
        severity = "critical"
        author = "security-suite"
    strings:
        $php = "<?php"

        $e1 = "eval(" nocase
        $e2 = "assert(" nocase
        $e3 = "preg_replace" nocase

        $src1 = "$_POST" nocase
        $src2 = "$_GET" nocase
        $src3 = "$_REQUEST" nocase
        $src4 = "php://input" nocase
    condition:
        $php and any of ($e*) and any of ($src*)
}

```

Read the condition as a sentence: *this is a PHP file **and** it contains an evaluation primitive **and** it contains a source of attacker-controlled input*. Each clause on its own is worthless. Together they describe the irreducible structure of a web shell identified in Chapter 2 — untrusted input reaching an execution primitive.

Three properties make this durable. It is **structural**, keying on what a web shell must do rather than how a particular one is written. It is **resilient to variation**, because the `$e` and `$src` families absorb alternatives without touching the condition. And it **fails safely**: a PHP file with `eval` but no request input does not fire, which is correct, because that file is a template engine.

THE GENERALISABLE PATTERN

Capability plus context. Find the two things the attacker cannot avoid doing, and require both. This survives obfuscation better than any single-string rule, because obfuscation changes spelling while structure stays fixed.

It is also the reason to prefer several short strings in families over one long clever one. Long strings are brittle — one changed byte and you have a false negative you will never hear about.

The same shape, three more times

Once you have the pattern, the rest of the web shell ruleset writes itself. The ASPX variant pairs a page directive with a request accessor and an execution primitive:

```

rule ASPX_Webshell : webshell
{
    strings:
        $page = "<%@ Page" nocase
        $a1 = "Request.Item" nocase
        $a2 = "Request.Form" nocase
        $b1 = "System.Diagnostics.Process" nocase
        $b2 = "CreateObject" nocase
        $b3 = "ExecuteReader" nocase
    condition:
        $page and any of ($a*) and any of ($b*)
}

```

And the JSP variant, which is tighter still because the Java idiom for running a command is so specific:

```

rule JSP_Webshell : webshell
{
    strings:
        $jsp = "<%"
        $req = "request.getParameter" nocase
        $exec1 = "Runtime.getRuntime().exec" nocase
        $exec2 = "ProcessBuilder" nocase
    condition:
        $jsp and $req and any of ($exec*)
}

```

Three files, one idea, consistently applied. That consistency is worth as much as the individual rules: when every rule in a set is built the same way, the set is far easier to review, tune, and hand to someone else.

Anchoring and bounding

The entropy rule from Chapter 3 demonstrates two more precision techniques in one line:

```

condition:
    $mz at 0 and math.entropy(0, filesize) > 7.2 and filesize < 20MB

```

`$mz at 0` anchors position — the file *starts* with MZ, so it is a PE, not a text file mentioning MZ. `filesize < 20MB` bounds cost, because entropy over a 4 GB file is expensive and rarely informative. Between them they turn a vague heuristic into something you can afford to run on every file.

The 7.2 threshold deserves comment. English text sits around 4.5 bits per byte, normal executable code around 6.0 to 6.5, and compressed or encrypted data pushes toward the theoretical maximum of 8.0. The 7.2 line catches packed and encrypted content while leaving ordinary binaries alone. It also

catches every legitimate installer, self-extracting archive, and anything else compressed — which is why this rule is `severity = "medium"` and not higher. It is a *hint*, and the metadata says so.

Indicator rules and their honest limits

Some rules are just lists of known-bad strings:

```
rule Credential_Dumper_Indicators : malware
{
    meta:
        description = "Strings associated with LSASS credential dumping tools"
        severity = "critical"
    strings:
        $a1 = "sekurlsa::logonpasswords" nocase
        $a2 = "privilege::debug" nocase
        $a3 = "lsadump::sam" nocase
        $a4 = "mimikatz" nocase
        $a5 = "gentilkiwi" nocase
    condition:
        any of them
}
```

This is a fundamentally weaker rule than the web shell ones, and it is important to be clear-eyed about why. It matches *names*, not *behaviour*. Renaming defeats it. A recompiled variant with the command strings changed walks straight past.

So why ship it? Because unmodified public tooling remains extremely common, because the strings are distinctive enough that false positives are rare, and because it costs nothing to run. It is a cheap net for the careless majority. What it must not be is your *only* control against credential theft, and a ruleset that consists entirely of rules like this is far weaker than its rule count suggests.

A CAVEAT ON INDICATOR RULES

This rule will fire on security documentation, training material, detection rules themselves, and blog posts about credential dumping. In a SOC folder full of analysis notes it is nearly useless; in an FTP upload directory it is excellent. The same rule has different precision in different places — which is an argument for scoping rulesets to directories, not deploying one global set everywhere.

Behaviour rules: the strongest kind

Compare the dumper rule with this one:

```

rule Defense_Evasion_Commands : suspicious
{
    meta:
        description = "Shadow copy deletion or security tooling being disabled"
        severity = "critical"
    strings:
        $v1 = "vssadmin delete shadows" nocase
        $v2 = "wbadmin delete catalog" nocase
        $v3 = "bcdedit /set {default} recoveryenabled no" nocase
        $d1 = "Set-MpPreference -DisableRealtimeMonitoring" nocase
        $d2 = "netsh advfirewall set allprofiles state off" nocase
    condition:
        any of them
}

```

Structurally identical — `any of them` over a string list — but far more durable, because these strings are not names an attacker chose. They are **the command syntax the operating system requires**. To delete volume shadow copies you must invoke `vssadmin delete shadows`; there is no synonym. Renaming your tool does not help, because the string is not your tool's name, it is the API you are obliged to call.

That is the hierarchy worth internalising:

Rule keys on	Durability	Example
Tool name or author	Weakest — defeated by renaming	"mimikatz", "gentilkiwi"
Tool-specific syntax	Moderate — survives renaming, not rewriting	"sekurlsa::logonpasswords"
OS-required syntax	Strong — attacker cannot avoid it	"vssadmin delete shadows"
Structural requirement	Strongest — survives obfuscation	input source + execution primitive

Aim as far down that table as the target allows. Most rulesets are top-heavy because the top is easy to write, and that is why most rulesets are more fragile than their authors believe.

Metadata is a contract

One last property of the case study's rules, easy to overlook because it does not affect matching at all: every rule declares `severity` in its metadata, and the application reads it.

```

SEVERITY_BY_TAG = {
    "critical": "critical", "ransomware": "critical",
    "malware": "high", "webshell": "high", "exploit": "high",
    "suspicious": "medium", "pua": "low", "test": "info",
}

@staticmethod
def severity_of(meta, tags):
    declared = str(meta.get("severity", "")).lower().strip()
    if declared in SEVERITY_RANK:
        return declared
    for tag in tags:
        mapped = SEVERITY_BY_TAG.get(str(tag).lower())
        if mapped:
            return mapped
    return "medium"

```

Explicit `severity` wins; a tag mapping is the fallback; unlabelled rules default to medium. The consequence is that **the person who writes the rule sets its triage priority**, in the same file, at the same moment, with the threat fresh in mind — rather than an operator guessing weeks later from a rule name. The alternative, a severity table maintained separately from the rules, drifts out of sync within a month. Every time.

This is a small design decision with an outsized effect on how a ruleset ages, and it costs one line per rule.

PART THREE

Building the Sensor

Everything between a rule and an alert — and the four bugs that only appear once real files start arriving.

Anatomy of a Detector

A detection engine is the easy part. Everything around it — deciding what to scan, when to scan it, what to record, and who to tell — is the system.

Starting from a script

Security Suite began as a single file of about ninety lines. It is worth looking at what that script got right before dismantling it, because the instincts behind it were sound:

- **It used `yara-python`, not `subprocess`.** Shelling out to the `yara` binary means a process spawn and a rules recompile per file. In-memory matching against a pre-compiled ruleset is orders of magnitude faster.
- **It wrote NDJSON.** One JSON object per line, appended. This is the correct format for an event log — streamable, greppable, and ingestible by every SIEM without a parser.
- **It attempted correlation.** Most first attempts at a file scanner emit only the file verdict. Reaching for authentication context is the right instinct, even though the implementation was broken in a way Chapter 7 dissects at length.
- **It was honest about its fallback.** When no rules file existed it compiled a test rule rather than pretending to be armed.

What it lacked was not cleverness. It lacked the accumulated scar tissue of having run against a real filesystem — which is precisely the material of the next two chapters.

Decomposition

The rebuilt system splits into six modules, each with one job:

Module	Responsibility	Why it is separate
<code>config.py</code>	Defaults, <code>config.json</code> loading, directory creation	Every other module takes configuration as an argument, so all of them are testable with a synthetic config.
<code>engine.py</code>	Compile rules, scan bytes, compute hash and entropy, map severity	Pure function of input bytes. No I/O policy, no scheduling, no state. Easy to test in isolation.
<code>telemetry.py</code>	Collect recent authentication failures from the OS	Platform-specific and permission-sensitive. Isolating it means the rest of the system does not care which OS it is on.
<code>watcher.py</code>	Decide what to scan and when; record outcomes	All the timing policy lives here — the settle logic, the rescan rules, the exclusions.
<code>store.py</code>	Persist events, hold recent history, publish to subscribers	One place that owns durability and the live feed, so the monitor never talks to the HTTP layer.
<code>server.py</code>	HTTP API, static files, Server-Sent Events	Presentation. Removing it leaves a fully functional headless detector.

The load-bearing property here is that **the engine knows nothing about files arriving and the watcher knows nothing about YARA internals**. When the settle-timing bug in Chapter 6 needed fixing, the change was confined to `watcher.py`. When the match-detail format changed, only `engine.py` moved. Detection systems accumulate special cases for years; module boundaries are what stop those special cases from becoming a single thousand-line function nobody will touch.

The engine: compile once, scan many

Rule compilation is expensive; matching is cheap. So compilation happens once at startup, and the compiled object is held and shared:

```

def reload(self) -> dict:
    """Recompile rules. Bad files are reported, not silently swallowed."""
    started = time.perf_counter()
    errors, sources = [], {}

    for path in self._rule_files():
        namespace = path.stem
        try:
            text = path.read_text(encoding="utf-8", errors="replace")
            yara.compile(source=text) # validate individually for a precise error
        except (OSError, yara.Error) as exc:
            errors.append({"file": str(path), "error": str(exc)})
            continue
        sources[namespace] = text

    using_fallback = False
    if not sources:
        sources = {"builtin": FALLBACK_RULE}
        using_fallback = True

    compiled = yara.compile(sources=sources)

```

The detail worth stealing is the double compile. Each rule file is compiled **on its own first**, purely to find out whether it is valid, and only then are the good ones combined. Compile them all at once and a single syntax error in one file fails the whole batch with a message that may not even name the offending file — so an operator who adds a broken rule loses their entire ruleset and gets an unhelpful error. With per-file validation, one bad file is reported by name and the other twelve rules keep working.

FAIL LOUDLY, DEGRADE GRACEFULLY

These two principles look contradictory and are not. *Degrade gracefully*: one broken rule file must not take down detection. *Fail loudly*: the operator must know, in the interface, that a file is broken and which one.

The original script did the first without the second — it caught the compile error and quietly fell back to a dummy rule. A detector that has silently disarmed itself while continuing to report green is worse than one that has crashed, because nobody investigates a healthy-looking dashboard.

Enrichment at scan time

A match tells you a rule fired. It does not tell you what the file was. The scan path attaches identity and shape:

```
def scan_file(self, path: str) -> dict:
    stat = os.stat(path)
    if stat.st_size > self.max_file_bytes:
        return {"file_path": str(path), "skipped": "file larger than max_file_mb",
                "file_size": stat.st_size, "matches": [], "severity": None}

    started = time.perf_counter()
    with open(path, "rb") as handle:
        head = handle.read(1 << 20)          # first MiB drives entropy + preview
    raw_matches = rules.match(filepath=str(path), timeout=60)
    result = self._result(str(path), head, raw_matches, started, stat.st_size)
    result["sha256"] = sha256_of(path)
    result["file_name"] = os.path.basename(path)
    result["modified"] = time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(stat.st_mtime))
    return result
```

Four pieces of enrichment, each earning its cost:

- **SHA-256** is the file's identity. It is what you paste into VirusTotal, what you search your estate for, and what tells you that the thing in `uploads/` is the same thing that appeared on a workstation last Tuesday. Without a hash, an alert refers to a path that may not exist tomorrow.
- **Size** bounds cost and provides context. A 43-byte web shell and a 40 MB one are different stories.
- **Entropy** over the first megabyte hints at packing or encryption. Computed on a bounded prefix, not the whole file, so it stays cheap.
- **Modification time** is the beginning of a timeline. Combined with the scan timestamp it tells you how long the file sat before anyone noticed.

Note the size guard runs *before* anything is read. A detector that tries to hash a 200 GB file because nobody bounded it is a detector that becomes an outage.

Recording the match, not just the verdict

The original script recorded matches as `[str(m) for m in matches]` — a list of rule names. Everything else YARA returns was discarded. What the analyst needs is the evidence:

```

for match in raw_matches:
    meta = dict(match.meta)
    tags = list(match.tags)
    strings = []
    for sm in match.strings:
        for instance in sm.instances[:5]:
            strings.append({
                "identifier": sm.identifier,
                "offset": instance.offset,
                "preview": printable(instance.matched_data),
            })
    matches.append({
        "rule": match.rule, "namespace": match.namespace, "tags": tags, "meta": meta,
        "severity": self.severity_of(meta, tags),
        "description": meta.get("description", ""),
        "strings": strings[:12],
    })

```

The difference in triage is stark. `Ransom_Note_Template` tells the analyst a rule fired. `$a1 at 0x0: "Your files have been encrypted"` and `$b1 at 0x2b: "bitcoin"` tells them *what the file actually says*, and that is usually enough to reach a verdict without ever opening it — which matters, because opening suspicious files is how analysts become incidents.

Two bounds keep this from becoming a liability: at most five instances per string, at most twelve strings per match. A rule matching a string that occurs fifty thousand times should not produce a fifty-thousand-element JSON array. And `printable()` renders bytes as safe ASCII with dots for non-printables, so binary content cannot corrupt the log or the terminal rendering it.

The store: durability, history, and a live feed

`store.py` owns three concerns that look separate and are not: what gets written to disk, what stays in memory, and who gets told.

```

def add(self, event: dict, persist: bool = True) -> dict:
    event.setdefault("id", new_id())
    event.setdefault("timestamp", now_iso())
    event.setdefault("status", "new")
    event.setdefault("_epoch", time.time())
    with self._lock:
        if persist:
            try:
                with self.path.open("a", encoding="utf-8") as handle:
                    handle.write(json.dumps(event, default=str) + "\n")
            except OSError as exc:
                print("[-] Could not persist finding: " + str(exc))
        self._events.append(event)
        self._count(event)
        dead = []
        for sub in self._subscribers:
            try:
                sub.put_nowait(event)
            except queue.Full:
                dead.append(sub)
        for sub in dead:
            self._subscribers.remove(sub)
    return event

```

The `persist` flag is a deliberate policy decision. **Matches are written to disk; clean scans are not.** A directory with 50,000 benign files would otherwise produce 50,000 lines of `"verdict": "clean"` per sweep, and `findings.ndjson` would become an audit trail of every file ever seen rather than a log of things that need attention. Clean scans still live in the in-memory ring buffer, so the dashboard can show throughput — they simply do not consume disk forever.

The subscriber loop is the live feed. Each connected browser holds a bounded queue; a slow or dead client fills its queue and is dropped rather than blocking the scanner. This is the correct priority: a detector must never stall because a dashboard tab is unresponsive.

Why the ring buffer matters

`collections.deque(maxlen=2000)` bounds memory regardless of uptime. On startup, the tail of the NDJSON file is replayed into it:

```

with self.path.open("r", encoding="utf-8", errors="replace") as handle:
    tail = deque(handle, maxlen=self.history_limit)

```

`deque(file_object, maxlen=N)` reads the file and keeps only the last N lines, without holding the whole file in memory. It is a neat idiom and exactly right here: restarting the service does not empty the dashboard, but a 4 GB findings log does not blow up the process either.

Choosing not to use a framework

The HTTP layer is `http.server` from the standard library rather than FastAPI or Flask. That is unusual enough to justify.

A detection tool is often deployed onto a machine that is already having a bad day — an incident response context, a locked-down server, a host with no outbound internet to reach a package index. Every dependency is a thing that must be installed, version-pinned, audited, and kept patched. The suite has exactly one hard dependency, `yara-python`, because that one is doing irreplaceable work. The web framework was not: about 240 lines of `http.server` provides routing, JSON responses, static files, and Server-Sent Events.

This is not an argument against frameworks in general. It is an argument that **security tooling has a different dependency calculus than product code**, because it needs to run in degraded environments and because its supply chain is itself a target.

The Race Conditions Nobody Warns You About

Four bugs, all invisible in testing, all discovered by running the thing against a real filesystem. Each one is a category, not an incident.

This is the most useful chapter in the book, because none of these problems appear in tutorials. They appear the first week you run a scanner for real, and each one degrades detection silently.

Bug one: the half-written file

The naive monitor is the one everybody writes first:

```
seen_files = set(os.listdir(WATCH_DIRECTORY))
while True:
    time.sleep(2)
    current_files = set(os.listdir(WATCH_DIRECTORY))
    for file_name in current_files - seen_files:
        scan(os.path.join(WATCH_DIRECTORY, file_name))
    seen_files = current_files
```

A file appears in a directory listing the moment it is *created*, not when it is finished. Upload a 50 MB archive over a slow connection and the filename exists within milliseconds while bytes keep arriving for the next thirty seconds. The scanner sees a new name, scans what is there — perhaps 2 MB of a 50 MB file — finds nothing, marks it seen, and never looks again.

The failure is silent and total. No error, no warning, a clean verdict on a file that was never fully examined. In testing you will never see it, because test files are small and written instantly.

The fix: settle detection

A file is scanned only once its size and modification time have stopped changing for a configured interval:


```

signature = (stat.st_mtime, stat.st_size)
if self._scanned.get(path) == signature:
    continue                                # unchanged since the last scan

previous = self._pending.get(path)
if previous is None or previous[:2] != signature:
    # New or still being written: restart the settle timer.
    self._pending[path] = (stat.st_mtime, stat.st_size, now)
    continue
if now - previous[2] < self.cfg.settle_seconds:
    continue                                # not settled yet

self._pending.pop(path, None)
self._scanned[path] = signature
self.scan_and_record(path, trigger="monitor")

```

Two dictionaries do the work. `_pending` tracks files still changing, with the timestamp at which the current signature was first observed. `_scanned` records the signature at the moment of the last scan. A file moves from pending to scanned only when it has been stable across at least one full poll interval plus `settle_seconds`.

Because `_scanned` stores the signature rather than just the path, a **modified** file is rescanned automatically — its new size or mtime will not match the stored signature. The naive version, storing only names in a set, could never notice a file changing after its first appearance. That is a second silent failure hiding inside the first.

VERIFYING A TIMING FIX

Timing bugs need timing tests. The case study's test writes a file in two pieces with a deliberate pause:

```
f.write("harmless start ") — flush — sleep 0.5s — f.write("malware") — flush
```

Then it asserts that **exactly one** event exists for that file and that it is a match. A premature scan would produce a clean verdict on the first fragment; a double scan would produce two events. One event, correct verdict, is the only passing outcome.

What the settle window does not guarantee

Settle detection is a heuristic, and it is worth stating its limit precisely rather than trusting it further than it deserves. A file is scanned once it has been stable for `settle_seconds`. If the writer *stalls* for longer than roughly `poll_interval + settle_seconds` — a network upload that pauses, a slow archive extraction, a process that blocks on something — the monitor concludes the file is finished and scans a partial file.

Building a test that forced exactly that (a five-second stall against a half-second poll and a half-second settle) showed what actually happens:

```
events for the stalled upload, in order:
    scan      clean
    yara_match critical  Defense_Evasion_Commands
```

The premature scan happens and produces a clean verdict. Then the writer resumes, the file's size and mtime change, the signature no longer matches what `_scanned` recorded, and it is scanned again — this time catching the content.

So the failure mode is **a spurious clean event, not a missed detection**. That is a considerably weaker failure than it first appears, and it is a direct consequence of storing the signature rather than the filename: the same decision that makes modified files get rescanned is what makes a prematurely-scanned file get a second look.

TEST THE INVARIANT, NOT THE IMPLEMENTATION

The original test for this asserted that a file written in two pieces produces *exactly one event*. That assertion passed, then failed, then passed again — it was racing the poll interval.

Counting events was testing an implementation detail. The property that actually matters is that the final verdict is correct and the alert is not duplicated. A test that fails intermittently is worse than no test: it trains you to re-run until green, which is how a real regression eventually gets waved through.

Bug two: the self-scan feedback loop

This one is delightful, in the way that only bugs happening to someone else are delightful.

The detector writes alerts to `findings.ndjson`. Each alert contains the matched strings — the evidence, as designed in Chapter 5. If that file happens to sit inside a watched directory, the scanner sees a new file, scans it, and finds the ransom-note strings it helpfully recorded there a moment ago. It alerts. The alert is appended to the same file. The file changes, so it is rescanned. It now contains two alerts' worth of malicious strings.

The output during discovery is unmistakable:

```
[!] ALERT CRITICAL Ransom_Note_Template in ../ss_test/note.txt
[!] ALERT HIGH PowerShell_Encoded_Command in ../ss_test/ps.txt
[!] ALERT CRITICAL Demo_TestKeyword, PowerShell_Encoded_Command,
    Ransom_Note_Template in ../ss_test/findings.ndjson
[!] ALERT CRITICAL Demo_TestKeyword, PowerShell_Encoded_Command,
    Ransom_Note_Template in ../ss_test/findings.ndjson
```

Every alert generating a further alert, each one accumulating more matched strings than the last. Left running, it fills the disk.

The fix, and the principle

```
# Never scan our own output: findings.ndjson contains the strings that
# tripped the rules, so scanning it would alert on itself forever.
self._self_paths = {
    os.path.normcase(os.path.abspath(p))
    for p in (cfg.findings_log, cfg.triage_file)
}
```

Checked on every candidate, with `normcase` and `abspath` so that `./data/findings.ndjson` and `C:\...\Data\Findings.NDJSON` compare equal on a case-insensitive filesystem.

The general principle is larger than this bug: **a detector's own artifacts are part of its environment**. Log files, quarantine directories, temporary extraction folders, and the rule files themselves all contain the strings the detector hunts. Any of them inside a monitored path creates a loop. It is the security-tool equivalent of audio feedback, and the fix is the same — keep the output away from the input.

THIS IS NOT A HYPOTHETICAL

Default configuration puts the findings log in `data/` and watches `uploads/`, so the shipped configuration is safe. The loop appeared the moment a test pointed both at one temporary directory — which is exactly what an operator does when they set `watch_paths` to a project root to "scan everything".

Bug three: the file that vanishes mid-scan

During testing, a real PHP web shell was written to disk to exercise the web shell rules. The scan raised an error, and the file was gone.

Windows Defender had quarantined it between the directory sweep and the scan — a window of a few hundred milliseconds. The detector had found a new file, decided to scan it, and by the time it called `os.stat()` the file no longer existed.

This is not an edge case. On any endpoint with antivirus, the detector is in a race with another security product over the same file, and it will lose that race regularly. Files also get moved by the process that uploaded them, deleted by cleanup jobs, or locked by a backup agent.

The fix: distinguish absence from failure

```
try:
    result = self.engine.scan_file(path)
except (FileNotFoundError, PermissionError) as exc:
    # Normal on a live endpoint: antivirus quarantined the file, or it
    # was moved/locked between the sweep and the scan. Not an error.
    return self.store.add({
        "event_type": "scan",
        "file_path": str(path),
        "file_name": os.path.basename(path),
        "skipped": "unavailable at scan time (" + type(exc).__name__ + ")",
        "trigger": trigger,
    }, persist=False)
except Exception as exc:
    return self.store.add({
        "event_type": "error", "severity": "low", ...
    })
```

A missing or locked file is recorded as a skipped scan, not an error. Genuine faults — a corrupt ruleset, a YARA timeout, an out-of-memory condition — still surface as errors. The distinction matters because an error count that ticks up constantly from routine AV activity is an error count nobody reads, and the one real fault hides inside it.

THE LESSON BEYOND THE BUG

Your detector is not the only security product on the host, and it is not the privileged one. It shares the filesystem with antivirus, EDR, backup agents, and indexers — all of which move, lock, and delete files without telling it.

Design for a hostile filesystem: every file operation can fail, the file can disappear at any point, and none of that is exceptional. It is Tuesday.

Bug four: forgetting to forget

The last one is subtle. The naive monitor accumulates seen filenames forever. Delete a file and re-upload one with the same name, and it is never scanned again — its name is still in the set.

That is trivially exploitable if an attacker knows a filename that has already passed inspection, and it also breaks ordinary workflows where a build job writes `artifact.zip` repeatedly.

```
# Forget files that disappeared so a re-upload is scanned again.
for gone in [p for p in self._scanned if p not in seen_now]:
    self._scanned.pop(gone, None)
for gone in [p for p in self._pending if p not in seen_now]:
    self._pending.pop(gone, None)
```

Tracking state about a mutable world means the state must be reconciled with reality on every pass, in both directions — additions *and* removals. Most implementations handle additions only, because additions are what the feature is about.

The baseline decision

One design choice sits alongside these bugs. When the monitor starts and the watched directory already contains ten thousand files, should it scan them?

```
if not self.cfg.scan_existing_on_start:
    # Baseline: remember what is already there so we only alert on new work.
    for path, stat in self._iter_files():
        self._scanned[path] = (stat.st_mtime, stat.st_size)
```

The default is no: record what exists and alert only on what arrives afterwards. Scanning everything on every restart means a service restart produces ten thousand events and buries anything current. The opposite behaviour is available behind `--scan-existing` for the deliberate case — a fresh deployment onto an existing server, where you genuinely want to know what is already there.

Neither default is universally right. What matters is that it is a decision, documented and switchable, rather than an accident of implementation.

Telemetry, Time, and Correlation

A file verdict answers "is this bad?" Correlation answers "how did it get here, and what else happened?" Getting the second one wrong is easy, and the failure is invisible.

A bug worth studying in detail

The original script attached authentication context to every alert. Here is the function, in full:

```
def get_recent_auth_events(self):
    """Tails auth.log for failed logins within the LOOKBACK_MINUTES."""
    events = []
    now = datetime.now()

    try:
        with open(AUTH_LOG_PATH, "r") as f:
            for line in f:
                if "Failed password" in line or "Invalid user" in line:
                    # Basic check: is this line from 'today'?
                    events.append({
                        "timestamp": now.isoformat(),
                        "raw_log": line.strip(),
                        "type": "auth_failure"
                    })
            return events[-5:] # Return only the 5 most recent for brevity
    except FileNotFoundError:
        return [{"error": "auth.log not accessible"}]
```

The docstring says it filters to a lookback window. It does not. Read what the code does:

- It iterates **every line in the entire log**, which may span months.
- It stamps each matched line with `now` — the moment of the scan, not the moment of the event. The real timestamp, sitting at the start of the raw line, is never parsed.
- It returns the last five entries **in file order**, which is not the same as the five most recent in time, and is not filtered by age at all.

The consequence: an alert could arrive claiming five authentication failures inside a five-minute window that actually happened last March. Every event in the output carries an identical, fabricated timestamp.

WHY THIS CLASS OF BUG IS DANGEROUS

It produces *plausible* output. The alert has a telemetry section, it is populated, the timestamps look reasonable, and nothing errors. An analyst reading it draws a conclusion — "brute force preceded this upload" — that the data does not support.

A bug that produces no output gets found on day one. A bug that produces confidently wrong output can survive for years and actively misdirect an investigation. In security tooling, fabricated context is worse than no context.

Timestamps are the whole problem

Correlation is a claim about time: *these things happened near each other*. If your timestamps are wrong, every correlation built on them is wrong, and confidently so. So the rebuilt telemetry layer does one thing above all — it parses the real timestamp from each source.

Syslog, and the missing year

The traditional syslog format has an irritating property: no year.

```
Sep 11 14:23:01 host sshd[1234]: Failed password for invalid user root from 10.0.0.9
```

Month, day, time. Parse it against the current year and every log line in January, read in December, lands eleven months in the future:

```
@staticmethod
def _parse_syslog_time(line: str) -> datetime | None:
    match = SYSLOG_RE.match(line)
    if not match:
        return None
    month = MONTHS.get(match.group("mon"))
    if not month:
        return None
    now = datetime.now()
    hour, minute, second = (int(p) for p in match.group("time").split(":"))
    try:
        when = datetime(now.year, month, int(match.group("day")), hour, minute, second)
    except ValueError:
        return None
    # Syslog omits the year: a timestamp in the future means last year.
    if when > now + timedelta(days=1):
        when = when.replace(year=now.year - 1)
    return when
```

The one-day tolerance absorbs clock skew and timezone drift without misclassifying genuinely recent entries. The `ValueError` guard catches February 29th in a non-leap year — a real failure mode when the year is being guessed.

The file is also read from the end rather than the beginning, because only the tail can possibly fall inside a five-minute window:

```
size = os.path.getsize(path)
with open(path, "r", encoding="utf-8", errors="replace") as handle:
    handle.seek(max(0, size - 512 * 1024))
    if size > 512 * 1024:
        handle.readline()    # discard the partial first line
```

Seek to 512 KB from the end, discard the first (probably truncated) line, and read forward. On a multi-gigabyte auth log this is the difference between a correlation that costs microseconds and one that costs minutes — and a correlation slow enough to notice is one an operator will eventually disable.

The Windows Security log

Windows keeps authentication events in a structured event log rather than a text file, which is better in every way except discoverability. The events that matter:

Event ID	Meaning	Why it is worth correlating
4625	Failed logon	The primary brute-force and credential-stuffing signal.
4648	Logon with explicit credentials	Often lateral movement — a process using credentials other than the caller's.
4740	Account lockout	The loud end of a password-guessing campaign.
4771	Kerberos pre-auth failed	Domain-side equivalent of 4625; catches attempts against AD directly.
4776	Credential validation failed	NTLM validation failure, including local accounts.

Reading them is a backwards walk from the newest record, stopping as soon as an event falls outside the window:


```

handle = win32evtlog.OpenEventLog(None, "Security")
flags = win32evtlog.EVENTLOG_BACKWARDS_READ | win32evtlog.EVENTLOG_SEQUENTIAL_READ

while len(events) < self.max_events and scanned < 4000:
    records = win32evtlog.ReadEventLog(handle, flags, 0)
    if not records:
        break
    for record in records:
        scanned += 1
        generated = record.TimeGenerated
        when = datetime(generated.year, generated.month, generated.day,
                        generated.hour, generated.minute, generated.second)
        if when < cutoff:
            scanned = 4000      # everything older follows; stop the outer loop
            break
        event_id = record.EventID & 0xFFFFFFFF
        if event_id not in WATCHED_EVENT_IDS:
            continue

```

Two Windows-specific details. `EVENTLOG_BACKWARDS_READ` starts at the newest record, so the loop can stop early instead of reading the whole log. And `record.EventID & 0xFFFFFFFF` masks off the severity and facility bits that Windows packs into the high bits of the event ID — compare the raw value against 4625 and it will never match, which is a classic half-hour of confusion.

Permission as a first-class state

Reading the Security log requires a privilege an ordinary user does not hold. The naive handling is a bare `except` that returns an empty list, which is indistinguishable from "no failed logons occurred" — a dangerous ambiguity, since one means *nothing happened* and the other means *you are blind*.

```

except pywintypes.error as exc:
    # 5 = access denied, 1314 = SeSecurityPrivilege not held.
    denied = exc.winerror in (5, 1314)
    return {
        "status": "denied" if denied else "error",
        "source": self.source,
        "detail": ("Access denied reading the Security log - relaunch the suite as "
                  "Administrator to enable logon correlation."
                  if denied else str(exc)),
        "events": [],
    }

```

Every telemetry result carries an explicit `status: ok, denied, unavailable, or error`. The dashboard renders it, and an alert produced while telemetry was denied says so in the finding itself. The analyst therefore knows the difference between *there were no failed logons* and *I could not see whether there were failed logons*, which is one of the more consequential distinctions in the whole system.

WINDOWS ERROR 1314

Access denied is error 5, and everybody handles it. Reading the Security log without the privilege actually returns **1314**, `ERROR_PRIVILEGE_NOT_HELD`, which a handler checking only for 5 reports as a generic error.

The difference is user-visible: a generic error message says "something went wrong", while a denial message says "run as Administrator" — one of which the operator can act on.

Portability is a telemetry problem

Detection itself is portable without effort. YARA matches bytes, a file is a file, and the scanner needs nothing from the operating system beyond `open()`. Telemetry is where portability actually costs something, because every platform records authentication failures somewhere different, in a different format, behind a different permission.

The case study's first attempt at this drew the line in the wrong place:

```
result = self._collect_windows() if IS_WINDOWS else self._collect_syslog()
```

Windows, and *everything else* reads `/var/log/auth.log`. That is wrong twice over. **macOS has no** `/var/log/auth.log` — it has used unified logging since 10.12, and the file the code was opening simply does not exist. And a large fraction of modern Linux distributions ship `journald` with no text auth log either; on Fedora, Arch, and RHEL the path is absent or vestigial.

On both platforms the adapter opened a file that was not there, reported `unavailable`, and did so forever. The failure was quiet, permanent, and looked exactly like a correctly-configured host with nothing to report.

A chain, not a branch

The fix is to stop asking *which platform is this* and start asking *what can I actually read here*:

```
def providers(self) -> list:
    """Ordered (name, callable) pairs to try for this platform."""
    if IS_WINDOWS:
        return [("windows_security_log", self._collect_windows)]
    if IS_MACOS:
        return [
            ("macos_unified_log", self._collect_macos),
            ("auth_log", self._collect_syslog),
        ]
    # Linux and other POSIX: prefer a text log, fall back to journald.
    return [
        ("auth_log", self._collect_syslog),
        ("journald", self._collect_journald),
    ]
```

Each provider is tried in turn until one returns usable data, and **every attempt is recorded in the result** — not just the one that worked:

```
{ "platform": "linux", "source": "journald", "status": "ok",
  "attempts": [ {"provider": "auth_log", "status": "unavailable"},
                 {"provider": "journald", "status": "ok"} ] }
```

That trail is what makes the layer debuggable. An operator seeing an empty telemetry panel can tell at a glance whether the text log was missing and journald answered, or whether both were refused — and the second case is an access problem they can fix.

The chain inherits the honesty requirement

Chapter 7 has already argued that an empty result must distinguish *nothing happened* from *I could not look*. A provider chain makes that easy to get wrong, because the obvious implementation returns an empty success once it runs out of providers.

```
# Nothing worked: return the first provider's diagnosis, not an empty
# success. An operator must be able to tell "quiet" from "blind".
result = first_usable or {"status": "unavailable", "source": self.source,
                          "detail": "no telemetry provider available",
                          "events": []}
```

And because a chain runs code from several sources, one of them failing badly must not take the poll loop with it:

```
try:
    result = collect()
except Exception as exc:
    # a provider must never crash the poll
    result = {"status": "error", "source": name, "detail": str(exc), "events": []}
```

A bare `except Exception` is usually a smell. Here it is the point: the caller is a monitoring loop whose job is to keep running, and an unanticipated failure in one optional enrichment source is not a reason to stop watching files.

Matching messages across four vocabularies

The original matcher looked for `"Failed password"` and `"Invalid user"` — `sshd`'s wording, and only `sshd`'s. `sudo`, `PAM`, `loginwindow` and `opendirectoryd` each phrase failure differently, so the marker list widened:

```
FAILURE_MARKERS = (  
    "failed password", "invalid user", "authentication failure",  
    "failed to authenticate", "authentication failed", "failed publickey",  
    "incorrect password", "auth failure", "failed keyboard-interactive",  
    "sudo: pam_unix", "failed login",  
)
```

Lowercased once and compared case-insensitively, because the same daemon capitalises inconsistently across versions. This is the least elegant part of the module and the most load-bearing: a marker list that misses a phrasing produces silence, and silence is indistinguishable from safety.

WHAT COULD NOT BE TESTED

The macOS and `journald` providers were written and unit-tested on a Windows machine, against stubbed `log show` and `journalctl` output — time windowing, failure filtering, account and IP parsing, permission errors, missing binaries, and chain fallback all verified with synthetic data.

Neither has been run against a real macOS or `systemd` host. The parsing is tested; the operating system interaction is not. That distinction belongs in the commit message and the release notes, because "cross-platform" claimed without it is the same species of overreach as a source card that reports online without making a request.

Choosing a window

The lookback window is the correlation's central parameter, and there is no universally correct value.

Window	Catches	Costs
1 minute	Only tightly coupled events — a logon immediately followed by an upload.	Misses a patient attacker; misses anything behind a queue or a slow transfer.
5 minutes	Interactive attack sequences: authenticate, upload, execute.	A busy server will show unrelated logon noise in every window.
1 hour	Slower campaigns and staged intrusions.	On any multi-user system, everything correlates with everything. The signal drowns.

Five minutes is the default because it approximates the timescale of a human driving a session. It is exposed as `lookback_minutes` precisely because the right answer depends on how busy the host is — on a single-purpose upload server, an hour may be perfectly informative; on a terminal server with two hundred users, even one minute will be noisy.

What a correlated alert actually claims

This deserves stating plainly, because correlation is routinely over-read.

When Security Suite attaches three failed logons to a web shell detection, it is claiming exactly one thing: **these events occurred within N minutes of each other on this host**. It is not claiming they are causally related. On a server exposed to the internet, failed SSH logons arrive continuously from background scanning; they will correlate with everything, including entirely benign events.

So correlation is a **prompt**, not a **conclusion**. It tells the analyst where to look next: same source IP as a recent successful logon? Same account that owns the upload directory? A burst against one account rather than scattered noise? Those questions turn temporal coincidence into a hypothesis. The tool supplies the coincidence; the analyst supplies the reasoning.

Tools that blur this — that present correlation as attribution — produce analysts who trust the machine's story over the evidence. Presenting telemetry as a labelled, timestamped, status-carrying list rather than a verdict is a deliberate choice to keep the human in the loop where the reasoning actually happens.

PART FOUR

Operating the Detector

Severity, attention, measurement, and the things that only break in production.

Severity, Triage, and the Economics of Attention

Analyst attention is the scarcest resource in security operations. Every design decision downstream of a rule is really a decision about how to spend it.

A detector that generates 500 alerts a day does not have 500 alerts of value. It has however many an analyst can actually judge — perhaps 40 — and 460 opportunities to train that analyst to stop reading. This chapter is about the machinery that decides which 40.

A severity scale that means something

Severity scales are usually invented in a meeting and then applied inconsistently forever, because nobody wrote down what the levels mean. Here is a definition worth stealing, phrased in terms of *response* rather than *badness*:

Level	Means	Expected response
critical	Confirmed malicious capability present.	Investigate now. Assume compromise until disproven. Web shells, credential dumpers, ransom notes, shadow copy deletion.
high	Strong indicator of attacker tradecraft; benign explanations exist but are uncommon.	Investigate this shift. Encoded PowerShell, download cradles.
medium	Suspicious pattern with plausible benign causes.	Review in batch. Packed executables, double extensions.
low	Weak signal; useful as corroboration, not on its own.	Do not alert. Query it when investigating something else.
info	Test and calibration rules. Confirms the pipeline works.	Never alert. Dashboard only.

The column that makes this usable is the third one. "Critical means very bad" is unfalsifiable and therefore unenforceable. "Critical means someone stops what they are doing" is a claim you can check against reality — and if your critical alerts do not actually cause anyone to stop what they are doing, they are not critical, whatever the metadata says.

THE SEVERITY INFLATION DEATH SPIRAL

Rule authors want their rules taken seriously, so they label them high. Soon most rules are high, so high stops meaning anything, so the next author labels theirs critical. Within a year everything is critical and the scale carries no information.

The defence is the response column. If every rule is critical, then by definition every alert requires immediate investigation — an obviously false claim, and one you can now point at in review.

Severity belongs to the rule

Chapter 4 established the mechanism; here is why it is an operational decision rather than an implementation detail.

The person writing a rule has the threat model in their head at that moment. They know whether this is a confirmed-bad indicator or a heuristic, what the false positive profile looks like, and what they would want to happen when it fires. That knowledge is at its peak while the rule is being written and decays immediately afterwards.

Storing severity anywhere other than the rule file guarantees drift. A separate mapping table, a database column, a dashboard configuration — each is a second thing that must be updated when a rule changes, and within a month someone will add a rule and forget. Putting `severity = "critical"` three lines above the condition makes the two impossible to separate: they move together, they get reviewed together, and they get deleted together.

The fallback chain matters too. Explicit metadata wins; a tag mapping catches rules imported from elsewhere; the final default is medium rather than info, so an unlabelled rule is visible rather than silently ignored. **Defaults should fail toward attention, not away from it.**

Triage states

An alert is not a notification. It is a small unit of work with a lifecycle, and a system without states is a system where every alert is permanently open:

State	Meaning
<code>new</code>	Nobody has looked. The only state that counts toward the open-alert metric.
<code>acknowledged</code>	A human has seen it and is working it. Prevents duplicate effort in a team.
<code>resolved</code>	Investigated and handled. Real, and dealt with.
<code>false_positive</code>	Investigated and wrong. This is a rule defect, not an analyst outcome.

That last row carries the weight. A false positive is feedback about detection content. If they are recorded as a disposition and never looked at again, the rule that produced them keeps producing them forever and nothing improves. The state exists so the tuning loop from Chapter 1 has an input — which is why the case study attaches an optional note when marking one, and persists it:

```
def set_status(self, event_id: str, status: str, note: str = "") -> dict | None:
    with self._lock:
        target = next((e for e in self._events if e.get("id") == event_id), None)
        if target is None:
            return None
        target["status"] = status
        target["triaged_at"] = now_iso()
        target["triage_note"] = note
        self._triage[event_id] = {"status": status, "at": target["triaged_at"], "note": note}
        self.triage_path.write_text(json.dumps(self._triage, indent=2), encoding="utf-8")
        return dict(target)
```

Triage state lives in a separate file from the findings log, and that separation is deliberate. `findings.ndjson` is **append-only and immutable** — the record of what the detector observed, which must never be rewritten. Triage is **mutable human judgement about** those events. Mixing them would mean rewriting an evidentiary log every time somebody clicks a button, which is exactly the property you do not want in something that might end up in an incident report.

The only KPI that matters

Six numbers sit across the top of the dashboard: files scanned, detections, open alerts, critical/high count, correlated auth failures, uptime. Only one of them is a health metric.

Files scanned is a throughput number. It proves the sensor is alive and says nothing about whether it is effective. **Detections** measures how much your rules fire, which is a property of your ruleset and your traffic, not of your security posture — a spike may mean an attack or a new benign file type.

Open alerts is the real one. It is the size of the queue of things a human has not yet judged, and it is the number that tells you whether the system is sustainable. If it trends up, one of three things is true: you are under attack, your rules are too noisy, or you do not have enough analyst time. All three demand action, and none of the other five numbers will tell you.

A RULE OF THUMB

If open alerts exceed what one person can triage in one shift, the detector is no longer a control — it is a backlog generator. The fix is almost never "hire more analysts". It is to tune the rules that produce the most alerts with the fewest true positives.

The dashboard's *top rules* panel exists for exactly this. It ranks rules by hit count, and the rule at the top of that list on a quiet week is the rule to examine first.

Designing the alert for the reader

A triage decision is a reading task under time pressure. The interface should answer, in order: **how bad, what file, which rule, why did it match, what else was happening.**

The case study's detail view does this literally — verdict and severity first, then file identity with hash and entropy, then each matched rule with its description, then the matched strings with offsets, then the correlated telemetry with its status. An analyst reaches a verdict by reading down the page and stopping when they have enough, which is usually at the matched strings.

The alternative — a table of rule names requiring the analyst to go and find the file themselves — moves the work from the machine to the human, and multiplies it by the number of alerts. Enrichment is computed once per detection; investigation is repeated per analyst per alert. That asymmetry is the entire argument for spending scan time on context.

Measuring and Tuning a Ruleset

A ruleset that is never measured is a ruleset that is slowly getting worse, and you will not find out until it fails to catch something.

Precision and recall, in the terms that matter

Two numbers describe every detection, and they trade against each other:

- **Precision** — of the things that fired, what fraction were real? Low precision means wasted analyst time and, eventually, ignored alerts.
- **Recall** — of the real things, what fraction fired? Low recall means missed intrusions.

The hard asymmetry is that **you can measure precision and you cannot measure recall**. Precision falls out of your triage data: count the alerts, count how many were marked false positive, divide. Recall requires knowing about the attacks you missed, which by definition you do not. This is why the triage states from Chapter 8 are not bookkeeping — they are the only precision instrument you have.

Recall has to be estimated by other means: red team exercises, retrospective hunting over stored artifacts, replaying known-malicious samples through the detector, and post-incident review asking bluntly *why did this not fire?*

Testing detection content

Detection code needs tests as much as any other code, and it needs a specific kind: tests that assert on the *number* of events, not merely their presence.

```
# The webshell ruleset, verified in memory
for label, sample, rule in [
    ("php-eval", b"<?php @eval($_POST['c']); ?>", "PHP_Webshell_Eval_Input"),
    ("php-exec", b"<?php system($_GET['cmd']); ?>", "PHP_Webshell_Command_Exec"),
    ("jsp", b'<% Runtime.getRuntime().exec(request.getParameter("c")); %>', "JSP_Webshell"),
    ("aspx", b'<%@ Page %> Request.Form System.Diagnostics.Process', "ASPX_Webshell"),
]:
    got = engine.scan_bytes(sample, label)
    names = [m["rule"] for m in got["matches"]]
    assert rule in names, (label, names)
    assert got["severity"] == "critical", got
```

Note that these samples are scanned **in memory**, never written to disk. That is not a stylistic choice — it is a direct consequence of Chapter 6's third bug. A real web shell written to a test directory gets quarantined by the host antivirus before the test can read it, so a disk-based test of web shell rules is flaky on precisely the machines where it matters most. Scanning bytes sidesteps the other security product entirely.

THE SAME THING HAPPENED TO THIS BOOK

An early draft of this chapter held that listing as a plain string in a source file. Windows Defender deleted the file mid-build — not the sample directory, not the detector, the *manuscript*.

The fix was to assemble the sample from fragments at build time, so no complete web shell literal ever exists on disk. If you write about detection content, you will eventually have to defend your own notes from your own antivirus.

Test the negative cases

Every ruleset needs benign samples that must *not* match, and they are more valuable than the positive cases. Positive tests tell you a rule works; negative tests tell you it has not quietly broadened into uselessness. The case study keeps `report_q3.txt` — ordinary business prose — in its sample set for exactly this reason, and the end-to-end test asserts the total detection count, so a rule that starts matching everything fails the suite loudly.

The tuning loop

When a rule produces a false positive, there are four responses, in rough order of preference:

1. **Add a required element.** The strongest fix. A rule firing on documentation about credential dumping gains a condition requiring PE structure, and the documentation stops matching while the binary still does.
2. **Narrow an existing element.** Add `fullword` to a short string, anchor a position, bound `filesize`. Cheap and often sufficient.
3. **Reduce the severity.** If the rule is genuinely a heuristic, say so. A medium-severity rule that batches for weekly review is far better than a high-severity one that gets ignored daily.
4. **Scope the rule.** Move it to a namespace applied only where it is precise. The credential-dumper rule is excellent on an FTP drop and useless in a folder of security research — same rule, different deployment.

Deleting the rule is a fifth option and sometimes the right one. A rule nobody acts on should not survive its first review.

Hot reload makes tuning cheap

The economics of tuning depend entirely on how long the loop takes. If changing a rule means restarting a service and losing state, tuning happens during maintenance windows — which means it does not happen.

```
elif route == "/api/rules/reload":
    info = self.ctx.engine.reload()
    self.ctx.store.broadcast({
        "event_type": "monitor", "timestamp": now_iso(),
        "message": "Reloaded " + str(info["rule_count"]) + " rules"
    })
    self._json(info)
```

Edit the file, click the button, watch the rule count and the compile time update. The whole loop is a few seconds, and validation errors appear in the interface rather than in a log nobody is tailing. Making the right thing fast is most of making it happen.

A worked example: finding a false positive with a book

Theory about precision is cheap. Here is the whole loop, run for real, with the manuscript of this book as the test corpus.

Scanning the book's own source is an unusually good precision test, because a book about detection engineering is the hardest possible benign document: it quotes IOC strings deliberately. Chapter 4 predicted the result — indicator rules "will fire on security documentation, training material, detection rules themselves" — and they did. Twelve rules fired, verdict critical.

Eleven of those were expected. The book genuinely contains `sekurlsa::logonpasswords`, `vssadmin delete shadows`, and "Your files have been encrypted", so rules keyed on those strings are behaving correctly; the false positive is a property of scanning documentation, not a defect. The twelfth was different. It matched on the word **Autofill**.

```
rule Browser_Autofill_Wallet_File_Targeting : malware
{
    meta:
        severity = "critical"
    strings:
        $w1 = "wallet.dat" nocase
        $w4 = "Exodus" nocase
        $f1 = "Autofill" nocase
        $f2 = "Web Data" nocase
    condition:
        any of ($w*) or any of ($f*)
}
```

An `or` between two families of single strings, at critical severity. Three deliberately innocuous files confirmed the scale of it:

File	Contains	Verdict
<code>release_notes.txt</code>	"Autofill did not remember the shipping address"	CRITICAL
<code>reading_plan.txt</code>	"Genesis, Exodus, Leviticus"	CRITICAL
<code>web_data_plan.txt</code>	"web data warehouse migration"	CRITICAL

A Sunday school reading list raising a critical malware alert is the single-string trap from Chapter 4, shipped.

The obvious fix was the wrong one

The reflex is to change `or` to `and`. Resist it, and read the rule's own description first: *"Code enumerating browser autofill data **or** cryptocurrency wallet files."* The OR is intentional. A stealer that only touches wallets is still a stealer, and forcing AND would stop matching it — trading a false-positive problem for a false-negative one, which is the worse trade.

The defect was never the structure. It was that the individual strings were too generic to carry a critical severity alone. So the fix keeps the OR and repairs the strings, then adds the missing half of the capability-plus-context pattern:

Problem	Before	After
Bare product name	<code>"Exodus"</code>	<code>"Exodus\exodus.wallet"</code>
UI vocabulary	<code>"Autofill"</code>	<code>"autofill_profiles"</code> (the table in Chrome's Web Data)
Generic phrase	<code>"Web Data"</code>	<code>"\Web Data"</code> , plus <code>\Login Data</code> , <code>credit_cards</code>
No behaviour required	—	an access or exfil primitive must co-occur

condition:

```
// The OR between wallet and browser artifacts is deliberate - a
// wallet-only stealer should still match - but either family now has
// to appear alongside an access or exfiltration primitive.
(any of ($w*) or any of ($f*)) and any of ($c*)
```

The context clause carries the idea in one line: **naming an artifact is documentation; opening or copying it is theft.** `sqlite3_open`, `CopyFile`, `CreateFileW`, `shutil.copy`, `ReadFile`, `SELECT *` `FROM`, `multipart/form-data`.

Test both directions, every time

A fix verified only against the thing that was broken is half a fix. This one was checked against five benign samples — the three above, browser documentation mentioning MetaMask, and the book itself — and four stealer patterns: a wallet path copy, a browser database read, a `wallet.dat` grab, and a card-table exfil.

0 false positives, 0 false negatives. The book dropped from twelve firing rules to eleven, and every remaining one is matching a genuine IOC string the text quotes. That is the tuning loop from Chapter 1 completing one full turn: a false positive observed, the rule's intent re-read, the actual defect identified rather than the apparent one, and both directions verified before shipping.

Performance as a detection property

Scan time is recorded on every result, and it is not vanity instrumentation. A ruleset that grows slowly enough to notice will eventually be a ruleset that cannot keep up with file arrival — at which point the queue grows, scans fall behind, and detection becomes delayed rather than absent, which is harder to notice.

Typical figures from the case study on ordinary hardware: fifty-five rules across twelve namespaces compile in roughly 50 ms; a small text file scans in one to three milliseconds; the seven-file sample directory completes in about 26 ms end to end. When a scan starts taking hundreds of milliseconds, the usual culprits are an unbounded regex, a rule without a `filesize` guard, or a `xor` modifier applied enthusiastically.

WHERE TO LOOK FIRST

Regular expressions and the `xor` string modifier dominate YARA performance problems. Both expand the search space enormously. Before reaching for either, ask whether two or three plain text strings combined in the condition would express the same idea — they almost always will, and they will run an order of magnitude faster.

Converging with Vulnerability Intelligence

File detection answers "did something bad arrive?" Vulnerability intelligence answers "what here could be attacked in the first place?" They are different questions, and a console that shows both is more useful than either alone — provided you are honest about what joining them does and does not prove.

The case study grew a second data plane: a **source lattice** of vulnerability and reputation feeds, with three of them implemented as live adapters. This chapter builds the NVD adapter end to end, because it exercises every problem you meet when a detection tool starts talking to the outside world — pagination, rate limits, caching, credentials, and a TLS failure that had nothing to do with the remote service.

Two shapes of external source

The seven sources in the lattice are not equivalent, and the design turns on the difference:

Source	Kind	What it provides
NVD	live adapter	The National Vulnerability Database. A public REST API over ~390,000 CVE records. No credential required.
VirusTotal	live adapter	File and hash reputation. Requires an API key; every request is authenticated and rate limited per account.
OSV	live adapter	Open-source vulnerabilities, queried by commit, package, or purl. No credential required.
CISA KEV	catalog	Known Exploited Vulnerabilities — the subset actually being used in the wild.
GitHub Advisories	catalog	Advisory database covering the same ground from the repository side.
Vuls	bridge	An agentless scanner; the suite links out rather than embedding it.
ClawFire	reference	Fleet guardrails reference.

Catalog sources are links. That is a deliberate, defensible choice: an integration you have not built is more honestly represented as a link than as a stub that pretends to be an integration. **Live adapter** sources actually make network calls and report real state. The dashboard renders the distinction, so an operator can tell at a glance which panels carry data and which carry a hyperlink.

RESIST THE INTEGRATION FACADE

The most common failure in a security console is a panel that looks live and is not — a static card labelled "Threat Intel: Active" that has never made a request. It is worse than an empty panel, because it manufactures confidence.

If a source is a bookmark, label it as a bookmark. The lattice uses four honest states — `bridge`, `catalog`, `reference`, `optional` — and only promotes a card to `online` when a request actually succeeded.

The NVD CVE API

NVD publishes CVE data over a plain JSON REST interface:

```
https://services.nvd.nist.gov/rest/json/cves/2.0
```

Three properties shape any client you build against it.

It is offset paginated

The corpus is far too large for one response, so the API uses `startIndex` and `resultsPerPage` (maximum 2000). Every response echoes `totalResults`, and you page by advancing the offset until you have what you need:

```
while True:
    payload = self._get({
        "lastModStartDate": _stamp(start),
        "lastModEndDate": _stamp(end),
        "resultsPerPage": MAX_RESULTS_PER_PAGE,
        "startIndex": start_index,
    })
    total = payload.get("totalResults", 0)
    batch = payload.get("vulnerabilities", []) or []
    if not batch:
        break
    records.extend(normalise(v) for v in batch)
    start_index += len(batch)
    if start_index >= min(total, max_records) or len(records) >= max_records:
        break
```

Note `start_index += len(batch)` rather than `+= resultsPerPage`. If the server returns fewer records than requested — which it may — advancing by the requested size silently skips records. Advance by what you actually received.

It is rate limited, and the key is optional

This is the sharpest contrast with VirusTotal. NVD allows **5 requests per rolling 30 seconds without any credential**, and 50 with a free API key. The key raises a ceiling; it does not unlock access. VirusTotal, by contrast, authenticates every request — no key, no data at all.

That difference belongs in the architecture, not buried in a comment. An adapter that requires a credential must degrade to a clearly-labelled disabled state; one that does not should work out of the box:

```
# 5 requests / 30 s without a key, 50 / 30 s with one; stay under both.
self.limiter = RateLimiter(0.8 if self.api_key else 6.5)
```

A 6.5-second minimum interval is deliberately conservative — 5 per 30 s permits 6.0 — because the window is rolling and a burst at the boundary earns a 403. Syncing a three-day window took 57 seconds across two pages at that pace. With a key it would be a few seconds. Slow and reliable beats fast and throttled for a background sync.

Date windows are bounded

NVD rejects `lastModStartDate / lastModEndDate` ranges wider than 120 days, and wants timestamps with milliseconds and no timezone suffix. Both constraints are worth encoding rather than discovering in production:

```
MAX_WINDOW_DAYS = 120          # NVD rejects date ranges wider than this

def _stamp(when: datetime) -> str:
    """NVD wants ISO-8601 with milliseconds and no timezone suffix."""
    return when.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S.000")
```

A TLS failure that was not the server's fault

The first request to NVD failed:

```
URLError: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED]
certificate verify failed: certificate has expired (_ssl.c:1081)>
```

The obvious reading — NVD let a certificate lapse — was wrong. Checking the chain directly showed a perfectly valid Let's Encrypt certificate with three months left, and `curl` against the same URL returned HTTP 200 with a clean verification result. Python failed where curl succeeded, against the same host, on the same machine, at the same moment.

The cause was local:

```
>>> ssl.get_default_verify_paths()
cafile: None
capath: None
```

Python had no CA bundle of its own and was falling back to whatever the platform store held, which included an expired cross-signed root in the chain. `curl` ships its own current bundle, so it never consulted the stale one. The fix is to carry a current bundle rather than to trust the ambient one:

```
def ssl_context() -> ssl.SSLContext:
    """A verified TLS context, preferring certifi's bundle when installed."""
    try:
        import certifi
        return ssl.create_default_context(cafile=certifi.where())
    except Exception:
        return ssl.create_default_context()
```

THE WRONG FIX, AND WHY IT IS TEMPTING

Every search result for this error suggests `ssl._create_unverified_context()` or `verify=False`. It makes the error disappear instantly, and it silently converts an authenticated channel into an anonymous one — in a tool whose entire purpose is to tell you what to trust.

A security tool that disables certificate verification to make a request succeed has traded a real control for a convenience, and it will keep doing so long after anyone remembers why. Carry a current CA bundle instead. If verification genuinely cannot succeed, that is a finding, not an obstacle.

The general lesson is worth more than the fix. **When an outbound request fails, determine whether the failure is local before believing what it says about the remote end.** The error message named a certificate expiry; the certificate had not expired. One `curl` and one `openssl s_client` separated the two possibilities in under a minute.

Caching: what to keep and what to fetch

390,000 CVEs is roughly two gigabytes of JSON. Mirroring it locally would be slow, fragile, and pointless for a file detector. The adapter keeps two things instead:

- **A trailing window of recently modified CVEs** in `nvds/cves.ndjson`, refreshed by an explicit sync. This is the working set — what changed lately is what an operator is most likely to care about.
- **Individual lookups** in `nvds/lookups/<CVE-ID>.json`, written on first fetch. A CVE record is immutable enough for this to be safe, and it makes repeated lookups free.

Records are normalised on the way in. A raw NVD entry is deeply nested and carries several CVSS metric versions; the dashboard needs a flat shape, so the adapter flattens once at the boundary rather

than making every consumer walk the structure:

```
score, severity, vector, version = None, None, "", ""
for key in ("cvssMetricV40", "cvssMetricV31", "cvssMetricV30", "cvssMetricV2"):
    items = metrics.get(key) or []
    if not items:
        continue
    data = items[0].get("cvssData", {})
    score = data.get("baseScore")
    severity = data.get("baseSeverity") or items[0].get("baseSeverity")
    vector = data.get("vectorString", "")
    version = data.get("version", key)
    break
```

The ordering matters: newest CVSS version first, falling back through to v2. Records from 1999 carry only `cvssMetricV2`; recent ones may carry v3.1 and v4.0 together. Taking the first available in newest-first order gives the most current scoring each record actually has, and `CVE-1999-0095` still resolves to a score rather than a blank.

Mapping an external scale onto your own

CVSS severity and the suite's triage severity are different vocabularies serving different purposes, so they are mapped explicitly at the boundary:

```
SEVERITY_MAP = {
    # NVD CVSS severity -> suite severity
    "CRITICAL": "critical",
    "HIGH": "high",
    "MEDIUM": "medium",
    "LOW": "low",
    "NONE": "info",
}
```

The mapping happens to be one-to-one here, which makes it look like a formality. It is not. The moment a second source arrives with a four-level scale, or its own idea of what "high" means, having a named translation point means changing one dictionary rather than auditing every consumer. Chapter 8's argument about severity meaning a *response* applies across systems too: importing another organisation's severity labels without deciding what they mean in your queue is how a scale stops carrying information.

OSV: asking the other question

NVD and OSV look like the same kind of source and are not. The difference is which direction you approach them from, and it determines which one can answer a question you actually have.

	NVD	OSV
You supply	A CVE id, a keyword, or a date window	A thing you possess: a commit, a package and version, or a purl
You get back	Catalog records matching that query	The vulnerabilities affecting exactly that artifact
Good for	Browsing and enrichment — <i>what is CVE-X?</i>	Answering <i>is this specific thing vulnerable?</i>
Credential	Optional; raises the rate limit	None

That makes OSV the more natural companion to a file detector. When a scan turns up a dependency manifest, a vendored library, or a checked-out source tree, what you have is an *artifact*, not a CVE id. OSV converts the artifact straight into an answer:

```
curl -d '{"commit": "6879efc2c1596d11a6a6ad296f80063b558d5e0f"}' \
  "https://api.osv.dev/v1/query"
```

That commit belongs to HarfBuzz, and OSV returns three vulnerabilities affecting it — one of them identified only as [OSV-2020-484](#), a fuzzer-discovered heap overflow with no CVE at all. Which is the second reason to query OSV rather than NVD alone: **OSV carries vulnerabilities that were never assigned a CVE**, and a workflow keyed exclusively on CVE ids cannot see them.

Caching by query rather than by record

The NVD adapter caches records by CVE id, because a CVE id is a stable key. OSV has no equivalent — the unit of work is the *question*, not the answer — so the cache is keyed by a hash of the query itself:

```
def _cache_path(self, payload: dict) -> Path:
    key = hashlib.sha256(
        json.dumps(payload, sort_keys=True).encode("utf-8")
    ).hexdigest()[:24]
    return self.cache_dir / (key + ".json")
```

`sort_keys=True` is doing real work there. Without it, the same query built with a different insertion order serialises differently and hashes to a different file — producing a cache that silently never hits. Canonicalise before you hash, every time.

Refusing to invent a number

OSV records carry severity in up to three different places depending on which database contributed them, and sometimes only as a raw CVSS vector string such as

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H. It is tempting to parse that into a base score so every row gets a tidy number.

The adapter deliberately does not. Computing a CVSS base score correctly means implementing the full specification — exploitability and impact sub-scores, the scope-changed coefficients, and defined rounding behaviour. An approximation produces numbers that look authoritative and are wrong, which is the same failure as the fabricated telemetry timestamps in Chapter 7. The vector is passed through verbatim so an analyst can read it or feed it to a real calculator, and severity falls back to the contributing database's own word where one exists.

THREE SOURCES, THREE DIFFERENT SILENCES

Each adapter fails in its own way, and each says so distinctly. **NVD** without a key still works — it only goes slowly — so an empty cache means *not synced yet*, never *not permitted*. **OSV** needs no credential at all, so an empty result genuinely means *nothing known affects this artifact*. **VirusTotal** without a valid key returns nothing, so an empty result there might be *clean* or might be *rejected*, and only the explicit status separates them.

The same empty list on screen, three completely different meanings. This is Chapter 7's lesson again: the absence of a finding is informative only when you know the source was able to look.

Handling the credential

The VirusTotal adapter needs a real secret, and the repository is public. Three rules cover it, and all three are enforced in code rather than by discipline.

The key never enters the repository. It lives in `.env`, which is listed in `.gitignore`; `.env.example` is committed with an empty value to document the variable's existence. Loading needs no dependency:

```
def _load_local_env() -> None:
    """Load simple KEY=value pairs for local adapters without a dependency."""
    env_file = ROOT / ".env"
    if not env_file.exists():
        return
    for line in env_file.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        key, value = key.strip(), value.strip().strip("\"'")
        if key and key not in os.environ:
            os.environ[key] = value
```

Note that an existing environment variable always wins over the file. In a container or a CI runner the secret arrives through the environment, and a stale `.env` must not override it.

The key never enters `config.json`. The config object carries it as a field, but serialisation drops it — so a user who saves their configuration cannot accidentally write a credential into a file they later commit:

```
def to_dict(self) -> dict:
    values = asdict(self)
    # Credentials belong in the process environment, never in config.json.
    values.pop("virustotal_api_key", None)
    values.pop("nvd_api_key", None)
    return values
```

The key never reaches a client. The intel endpoint reports whether a key is configured and whether it works — never the value:

```
def _intel(self) -> dict:
    """Return source adapters without ever returning credentials to clients."""
    ...
    "configured": bool(vt_key) if source_id == "virustotal" else True,
```

`bool(vt_key)` rather than `vt_key` is the whole of it — one function call standing between a working dashboard and a credential leak to any page that can reach the API.

KEY STATUS IS NOT A BOOLEAN

"Configured" and "working" are different facts, and an operator needs both. The adapter probes the account endpoint and reports five distinct states: `online` (verified this minute), `configured` (present, unverified), `rejected` (401/403 — wrong or revoked), `rate limited` (429), and `offline` (unreachable).

`rejected` is the one that earns its keep. A revoked key and an absent key produce identical empty results, and only an explicit status distinguishes *nothing was found* from *I was not allowed to look* — the same distinction Chapter 7 drew about the Windows Security log.

When the API says no: capability tiers

VirusTotal's most powerful feature for a YARA-based tool is **Hunting** — Livehunt runs your rules against the live stream of files submitted to VirusTotal, Retrohunt runs them across roughly 600 TB of the past year's submissions, and VTDIFF helps generate rules from a set of hashes you already care about. For a detection engineer that is close to irresistible: the same rules, against everyone's samples instead of only your own.

Building it against the case study's key produced this:

Endpoint	Status	Meaning
/intelligence/hunting_rulesets	403	Livehunt unavailable
/intelligence/retrohunt_jobs	403	Retrohunt unavailable
/intelligence/search	403	Intelligence search unavailable
/files/<hash>	200	Hash lookup works

All of Hunting is an Enterprise feature. The public API tier cannot reach any of it, and no amount of code changes that. This is worth dwelling on because of what the tempting response would have been: build the Livehunt panel anyway, leave it empty, and label it "awaiting data". That is the integration facade from the start of this chapter, in its most seductive form — the feature is real, the code would be correct, and it would simply never work for this user.

Probe, do not assume

The adapter instead asks the API what it is allowed to do, once, and caches the answer:

```
CAPABILITY_PROBES = (  
    ("file_lookup", "/files/44d88612fea8a8f36de82e1278abb02f"), # EICAR, always present  
    ("livehunt", "/intelligence/hunting_rulesets?limit=1"),  
    ("retrohunt", "/intelligence/retrohunt_jobs?limit=1"),  
    ("intelligence_search", "/intelligence/search?query=type%3Apeexe&limit=1"),  
)
```

Two properties fall out of this that a hardcoded flag would not give you. The dashboard reports the **real** tier rather than a configured one, so an operator cannot mislabel their own deployment. And if the account is upgraded later, Hunting begins working with no code change and no redeploy — the probe simply returns a different answer.

The gated endpoints return **402 Payment Required** with the reason, which is an unusually apt status code: the request was well-formed, the caller is authenticated, and the only thing missing is a commercial relationship.


```
{
  "available": false,
  "feature": "livehunt",
  "tier": "public",
  "error": "livehunt requires a VirusTotal Enterprise account",
  "detail": "This key returns HTTP 403 for livehunt. Livehunt, Retrohunt and
            VTDIFF are Enterprise features; the public API tier cannot reach
            them."
}
```

What the free tier is actually good for

Hash reputation, which turns out to be the half that matters most for a file detector. The scanner already computes SHA-256 on every detection, so a lookup costs one request and returns the verdict of roughly seventy engines, a suggested family label, and a first-seen date. Against EICAR: 65 of 75 engines, label `virus.eicar/test`, first submitted in 2006.

Two constraints shape how it is wired in. The public tier allows **4 requests per minute and 500 per day**, so lookups are rate limited to match and cached by hash. And enrichment is deliberately **on demand rather than automatic** — auto-enriching every detection would exhaust a 500-request daily quota within minutes on a busy directory, and then fail silently for the rest of the day, which is the worst possible time for a reputation lookup to stop working.

THE LINE THIS CLIENT WILL NOT CROSS

VirusTotal accepts file uploads, and uploading would give far richer results than a hash lookup. The client never does it.

Submitting a file to VirusTotal **publishes** it — other users can download it. Auto-submitting whatever lands in a watched directory would eventually publish a customer's document, a configuration file containing credentials, or a database export, and there is no recall. Hash lookups send a fingerprint and nothing else.

Uploading is a legitimate action with real analytical value. It is a decision for a human who has looked at the file, not a default in a monitoring loop.

What CVE correlation actually claims

Chapter 7 warned that temporal correlation is a prompt, not a conclusion. Vulnerability correlation deserves the same caution, in a sharper form.

A CVE record says a vulnerability exists in some version of some software. It does **not** say that software is installed here, that the vulnerable version is the one running, that the vulnerable path is

reachable in this configuration, or that anyone has attempted to exploit it. A console showing 445 critical CVEs from the last three days is showing the state of the world, not the state of your estate.

That is precisely why the adapter syncs a window rather than claiming a posture. The honest workflow runs the other way — from a detection outward:

1. A YARA rule fires on an uploaded file, and the matched strings reference a product and version.
2. The analyst searches the CVE cache for that product to see what is known about it.
3. KEV tells them whether any of it is actually being exploited in the wild.
4. That context changes the triage decision — from "suspicious file" to "suspicious file targeting something we run, which is on the exploited list".

Step four is the only one that matters, and it is a human step. The machine's job is to make steps two and three take seconds instead of minutes. Building a system that instead emits "CRITICAL: 445 vulnerabilities detected" would be generating alert volume with no corresponding decision — precisely the pathology Chapter 8 is about.

From Detection to Pivot

An alert that names a bad file ends the machine's work and begins the analyst's. The question it leaves unanswered — *what do I hunt for now?* — is usually answerable from the same bytes the scanner already read.

Consider what the detector produced before this chapter's work. A ransom note lands in a watched directory, `Ransom_Note_Template` fires, and the alert carries the file path, the SHA-256, and the matched strings: "Your files have been encrypted", "bitcoin". All correct, all useless for the next step.

Because the analyst's actual next question is not *was this file bad*. They already know. It is: **which address demanded payment, what domain does the portal sit on, and has anything else on the estate touched them?** Those indicators were sitting in the same file, three lines from the strings that matched, and the detector threw them away.

Matched strings are not indicators

This distinction is worth being precise about, because the two look similar in an alert and serve opposite purposes.

	Matched strings	Extracted indicators
What they are	The evidence the rule fired on	Observables the file contains
Chosen by	The rule author, in advance	The attacker, incidentally
Answer the question	Why did this alert?	What do I search for next?
Change when	You edit the rule	The campaign changes infrastructure

A rule's strings are deliberately generic — that is what makes them match a family rather than one sample. "your files have been encrypted" is designed to hit ten thousand ransom notes. Searching your estate for that phrase finds nothing useful. Searching it for `1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa` finds every other machine the same actor touched.

What to extract

The case study pulls thirteen observable types out of any file that tripped a rule:

Category	Types
Network	URLs, domains, IPv4 addresses, .onion services
Financial	Bitcoin, Ethereum and Monero addresses
Contact	Email addresses
Host	Registry keys, Windows file paths
Reference	CVE identifiers, embedded MD5 and SHA-256 hashes

Financial addresses deserve their place. A wallet is the one indicator a ransomware operator genuinely cannot rotate cheaply — it is where the money has to arrive — and it links otherwise unrelated incidents to a single actor better than any domain.

Defang everything on the way out

Every extracted indicator is rendered unclickable before it is displayed:

```
http://185.220.101.47:8443/gate.php
becomes
hxxp://185[.]220[.]101[.]47:8443/gate[.]php
```

This is not decoration. Indicators get pasted into tickets, chat clients, wikis and spreadsheets, most of which auto-link anything resembling a URL. A live C2 link sitting inside an incident ticket is how somebody eventually clicks it — often on a machine with no proxy in front of it, often months later, often while demonstrating the incident to somebody else.

The raw value is kept alongside the defanged one for machine use, and the CSV export carries both. **Defanging is a display concern, not a storage one** — a pipeline that has to re-fang indicators before querying them has put the transformation in the wrong layer.

Noise is the whole problem

Extraction is easy. Extraction that an analyst will actually read is not, because most files are full of things that look like indicators and are not.

Schema hosts

Any XML, Office document, or SVG contains namespace declarations pointing at `w3.org`, `schemas.microsoft.com`, `purl.org` and friends. Extract naively and every Office document produces a dozen "indicators", none of which mean anything. A denylist handles these, and — this is the part easy to miss — **it has to apply to the host inside a URL, not just to bare domains**:

```
elif kind == "url":
    # The same denylist has to apply to the host inside a URL,
    # or every XML namespace declaration becomes an indicator.
    host = _host_of(value)
    if host and not _plausible_domain(host) and not host.endswith(".onion"):
        dropped["known_benign_url"] += 1
        continue
```

The first version of this extractor filtered `w3.org` as a domain and then happily reported `http://www.w3.org/2000/svg` as a URL. Two code paths, one rule, applied to only one of them.

Filenames that look like domains

`invoice.pdf`, `setup.exe`, `config.ini` — a domain regex matches all of these, because `.pdf` is structurally indistinguishable from `.co`. A suffix list of known file extensions removes them.

Addresses that are not interesting

Private and loopback addresses are a different case, and the instinct to drop them is wrong. `10.0.0.5` in a ransom note is noise; `10.0.0.5` in a lateral movement script is the finding. So they are **kept and labelled** rather than filtered:

```
def _classify_ip(value: str) -> str | None:
    addr = ipaddress.ip_address(value)
    if addr.is_loopback:
        return "loopback"
    if addr.is_link_local or addr.is_multicast or addr.is_unspecified or addr.is_reserved:
        return "reserved"
    if addr.is_private:
        return "private"
    return "external"
```

CHECK LOOPBACK BEFORE PRIVATE

In Python's `ipaddress` module, `127.0.0.1` satisfies `is_private` as well as `is_loopback`. Test `is_private` first and every loopback address is mislabelled as a private network address — a distinction that matters when you are asking whether a sample talks to the host it runs on or to the network around it.

The ordering of an if-chain over overlapping predicates is a design decision, not a formality.

Only extract from what matched

Thirteen regular expressions over a one-megabyte buffer is not free. Run that on every file a monitor sees and extraction dominates the scan budget, for no benefit — a clean file's observables are not

interesting, by definition.

```
if matches:
    severity = min((m["severity"] for m in matches), key=lambda s: SEVERITY_RANK[s])
    # Only extract for files that tripped a rule: a clean scan does not
    # need observables, and running 13 regexes over every file would
    # dominate the scan budget.
    iocs = extract_iocs(sample)
```

This is the same reasoning as the enrichment in Chapter 5 — SHA-256 and entropy are computed for every scan because they are cheap and identify the file, while anything expensive waits until there is a reason to care.

Aggregation is where the value appears

A single file's indicators are mildly useful. The same indicator appearing across several files is a campaign, and that is what the dashboard panel ranks on:

```
items.sort(key=lambda i: (-i["file_count"], -i["occurrences"], i["type"]))
```

File count first, occurrence count second. An address mentioned forty times in one dropper is less interesting than one mentioned once in each of four unrelated uploads — the second is evidence of a campaign, the first is evidence of a verbose author.

Each aggregated indicator carries first-seen and last-seen timestamps and the files it appeared in, which turns the panel into a small pivot table: *this wallet appeared in three uploads over two days, here they are.*

Export, because the pivot happens elsewhere

Whatever the panel shows, the actual hunt happens in a SIEM, an EDR console, or a firewall log search. So the indicators leave as CSV, carrying both the raw and defanged forms:

```
type,value,defanged,scope,occurrences,file_count,first_seen,last_seen
btc,1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa,1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa,,1,1,...
ipV4,194.26.29.156,194[.]26[.]29[.]156,external,1,1,...
```

A detection tool that cannot hand its findings to the next tool in the chain is a silo. The export exists so the analyst never has to retype an indicator — which is not only tedious but a genuine source of error, since a mistyped hash finds nothing and looks exactly like a clean result.

WHAT EXTRACTION DOES NOT DO

It does not tell you an indicator is malicious. A domain extracted from a malicious file might be the C2, or it might be the legitimate service the malware impersonates, or a string the author copied from Stack Overflow.

Extraction narrows *where to look* from an entire file to a dozen candidates. Deciding which of them matter is still the analyst's job, and a panel that presented these as confirmed-bad would be making exactly the overreach Chapter 7 warns about with correlation.

Running It for Real

Deployment is where design decisions get audited by reality.

Privileges: ask for the minimum, degrade honestly

Security Suite needs read access to the directories it watches. That is all — for detection. Authentication correlation additionally requires the privilege to read the Windows Security log, which in practice means Administrator.

The design decision is that **the elevated capability is optional and its absence is visible**. Run unprivileged and file detection works completely; the telemetry panel shows `denied` with an explanation naming the fix. Run elevated and correlation activates.

The pattern generalises: when a tool needs elevation for one feature, do not require elevation for the whole tool. Requiring more privilege than necessary is how security tools become the largest attack surface on the host — a scanner reading untrusted attacker-supplied files while running as SYSTEM is a genuinely alarming combination, and one that the industry has repeatedly been embarrassed by.

Deploying on each platform

Detection behaves identically everywhere. What changes is where telemetry comes from and what privilege it needs:

	Windows	macOS	Linux
Telemetry source	Security event log	unified log	auth.log, else journald
Privilege needed	Administrator	usually none	log read or <code>systemd-journal</code>
Launcher	<code>.lnk</code>	<code>.command</code>	<code>.desktop</code>
Optional dependency	<code>pywin32</code>	none	none

One command creates the right launcher for the host, with no elevation and nothing installed system-wide:

```
python install_shortcut.py      # create it
python install_shortcut.py --remove  # take it away again
```


Small platform details are worth encoding once rather than rediscovering: a Linux `.desktop` file must be executable or GNOME silently refuses to launch it; the desktop directory is localised and lives in `~/.config/user-dirs.dirs` rather than being `~/Desktop` everywhere; and only Windows wants an `.ico`, so the icon ships as SVG and PNG too.

The network boundary

The dashboard binds `127.0.0.1` and has no authentication. That combination is only safe because of the bind address, so the code defends it:

```
def _host_allowed(self) -> bool:
    """Block DNS-rebinding: only localhost names may talk to the API."""
    host = (self.headers.get("Host") or "").split(":")[0].strip("[]").lower()
    return host in ("localhost", "127.0.0.1", "::1", "")
```

Binding to loopback stops remote connections but does not stop **DNS rebinding**, where a malicious web page the operator visits resolves an attacker-controlled hostname to `127.0.0.1` and then issues requests to the local service from the browser. Those requests come from the local machine and pass the bind restriction. Validating the `Host` header rejects them, because the browser sends the attacker's hostname rather than a loopback name.

This is a small check that closes a real and frequently overlooked hole in local-only tools.

ON THE ON-DEMAND SCAN ENDPOINT

`POST /api/scan` will scan any path the running user can read, which is a broad capability — it is also the entire point of an on-demand scanner. The mitigation is not to cripple it but to keep the listener on loopback, validate `Host`, and never bind `0.0.0.0`. If this ever needs to be reachable remotely, it needs authentication and path restrictions first, in that order.

Feeding a SIEM

NDJSON exists so that other systems can consume it without a parser. One JSON object per line, append-only, no enclosing array to close:

```
# Everything critical from the log
cat data/findings.ndjson | jq -c 'select(.severity=="critical")'

# Just the hashes, for an estate-wide search
jq -r '.sha256' data/findings.ndjson | sort -u

# Which rules fire most
jq -r '.rule_names[]' data/findings.ndjson | sort | uniq -c | sort -rn

# Ship it
tail -F data/findings.ndjson | your-log-shipper --stdin
```

The append-only property is what makes `tail -F` work as an ingestion strategy, and it is why triage state was kept in a separate file. A log that gets rewritten in place cannot be tailed reliably — the shipper either re-sends everything or loses its position.

What actually breaks

Symptom	Usual cause
Detections stop appearing, no errors	The monitor thread died on an exception the loop did not catch, or the watched path became unreachable — a network share that dropped. Check <code>last_sweep</code> and <code>last_error</code> in the sensor panel; a stale <code>last_sweep</code> is the tell.
Alerts for files nobody uploaded	Another process is writing into the watched directory — a backup agent, an indexer, or the detector's own output. See Chapter 6.
Scan errors climbing steadily	Antivirus quarantining files mid-scan. Expected on endpoints; these should appear as skipped scans rather than errors, and an exclusion for the watched path may be appropriate.
Telemetry permanently empty	Not elevated, or the Security log is full and not rotating. Check the telemetry <code>status</code> field before assuming quiet.
Memory growth over weeks	History limit set very high, or a rule matching a string with enormous instance counts. Both are bounded in the shipped configuration; both come back if the bounds are raised.
Dashboard blank, API responds	The browser cached an old asset, or an SSE connection failed silently. The stream indicator in the header shows <code>reconnecting</code> when this happens.

Extending it

Three directions, in increasing order of effort:

More rules. Drop a `.yar` file into `rules/` and reload. Nothing else changes; severity and namespaces are picked up automatically. This is where nearly all the value is, and it requires no code.

More telemetry sources. `telemetry.py` exposes one method, `recent()`, returning a dict with `status`, `source`, and `events`. A process-creation source, a DNS source, or a firewall source implementing the same shape would appear in the detail view with no changes elsewhere. The interface was kept deliberately narrow for this reason.

Different actions on detection. The suite is read-only by design. Moving a matched file to quarantine is straightforward to add and should be approached with the full weight of Chapter 1's warning: the moment a detection takes a destructive action, its precision requirements change completely, and a false positive stops being an inconvenience and becomes an outage.

Closing

The detector in this book is about 2,300 lines of Python. The 73 rules are a few hundred lines of YARA. None of it is technically sophisticated, and that is the point worth leaving with.

What separates a detector that works from one that merely runs is not algorithmic cleverness. It is having noticed that files are written incrementally. That antivirus will delete your evidence. That syslog omits the year. That a log file full of matched strings placed in a watched directory will alert on itself forever. That an analyst reading your alert at 3 a.m. needs the matched string, not the rule name.

Those are not insights you can derive from documentation. They come from running the thing, watching it fail in a way you did not predict, and fixing the category rather than the instance. That process — not the rules, not the dashboard — is detection engineering.

The rule is the easy part. Everything that happens after it fires is the job.

— the whole book, compressed

APPENDICES

Reference

The complete ruleset, the HTTP API, every configuration key, and the command line.

The Complete Ruleset

73 rules across 15 namespaces. Each rule file becomes one YARA namespace named after the file, so rule names need only be unique within a file. Every rule declares `severity` in its metadata, which is what drives triage priority in the dashboard — see Chapter 8.

Namespace	Rules	Covers
<code>c2_network</code>	5	Command-and-control channels and reverse shells
<code>credential_theft</code>	5	Credential dumping, browser stores, keylogging
<code>cryptominer</code>	3	Mining configuration, flags, browser miners
<code>demo</code>	3	Pipeline verification and calibration
<code>info_stealer_rat</code>	5	Commodity stealers and remote-access trojans
<code>linux_threats</code>	7	Reverse shells, cron/systemd persistence, container escape
<code>office_macro_malware</code>	5	Auto-executing document macros and droppers
<code>phishing_social_engineering</code>	5	Credential harvesting and lure patterns
<code>ransomware</code>	5	Family indicators and extortion artifacts
<code>recon_persistence</code>	5	Discovery chains and persistence mechanisms
<code>rmm_tunnel_abuse</code>	5	Silent RMM installs, outbound tunnels, RDP exposure
<code>supply_chain</code>	6	Package install hooks, CI secret theft, pipeline tampering
<code>web_fraud_skimmer</code>	4	Payment card skimming and formjacking
<code>webshell</code>	4	PHP / ASPX / JSP server backdoors
<code>windows_threats</code>	6	PowerShell abuse, defence evasion, ransom notes

Severity distribution across the set: **32 critical, 30 high, 9 medium, 1 low, 1 info**. That is a top-heavy profile, and worth reviewing against Chapter 8's warning about severity inflation — a set where 85% of rules are critical or high should be able to justify each one against the "expected response" column.

c2_network — command and control

Rule	Severity	Catches
Cobalt_Strike_Beacon_Indicators	critical	Strings from Cobalt Strike beacon payloads and profiles
Reverse_Shell_OneLiner	critical	Reverse shell one-liners across scripting languages
DNS_Tunneling_Pattern	high	Excessive TXT/NULL lookups or base32-style subdomains
HTTP_C2_Beacon_Markers	high	HTTP clients built with fixed, unusual beacon headers
Known_C2_Framework_Strings	high	References to publicly known C2 frameworks

credential_theft — credential access

Rule	Severity	Catches
LSASS_Memory_Dump_Indicators	critical	Command or API patterns that dump LSASS memory
SAM_Registry_Hive_Dump	critical	Exporting the SAM / SYSTEM / SECURITY hives
Browser_Credential_Store_Access	high	Reading a browser's saved-password or cookie database
Keylogger_API_Hooking	high	Keyboard hooking or polling combined with local logging
Credential_Harvest_Clipboard_Config	high	Clipboard scraping or config scanning for stored secrets

info_stealer_rat — stealers and remote access

Rule	Severity	Catches
<code>Commodity_Stealer_Strings</code>	critical	Strings from common commodity infostealer families
<code>Screen_Webcam_Capture_With_Exfil</code>	critical	Capture APIs combined with network exfiltration
<code>Browser_Autofill_Wallet_File_Targeting</code>	critical	Enumerating autofill data or cryptocurrency wallets
<code>RAT_C2_Handshake_Strings</code>	critical	Fixed handshake or beacon strings from RAT families
<code>RAT_Persistence_Run_Key</code>	high	Run-key persistence plus remote-access behaviour

ransomware — extortion

Rule	Severity	Catches
<code>LockBit_Indicators</code>	critical	Strings and note filenames from LockBit
<code>Conti_Indicators</code>	critical	Strings and note filenames from Conti
<code>Sodinokibi_REvil_Indicators</code>	critical	Strings from Sodinokibi / REvil
<code>Cryptocurrency_Wallet_Near_Extortion_Text</code>	critical	A wallet reference alongside encryption-themed text
<code>Ransom_Note_Filename_Reference</code>	high	Code writing a file using a known ransom note filename

office_macro_malware — document delivery

Rule	Severity	Catches
Macro_AutoExec_Shell_Launch	critical	Auto-running macro trigger plus a shell or script launch
Macro_Template_Dropper	critical	Macro writing an embedded payload to disk before execution
Macro_Launches_PowerShell	critical	Office macro invoking PowerShell, typically encoded
Obfuscated_VBA_String_Building	high	VBA building strings character-by-character to evade scanning
Suspicious_AutoExec_XML_OLE	high	OLE/XML metadata carrying an auto-executing macro

recon_persistence — footholds

Rule	Severity	Catches
WMI_Event_Subscription_Persistence	critical	WMI permanent event subscription as fileless persistence
Scheduled_Task_Registry_Run_Persistence	high	Scheduled task or Run-key creation for persistence
Service_Creation_For_Persistence	high	A new Windows service running an attacker binary at start
LOLBin_Chaining	high	Living-off-the-land binaries chained to proxy execution
Discovery_Command_Chain	medium	Multiple host/network discovery commands chained together

web_fraud_skimmer — payment fraud

Rule	Severity	Catches
<code>JS_Payment_Card_Skimmer</code>	critical	JavaScript reading card fields and exfiltrating them
<code>Formjacking_Exfil_Domain_Pattern</code>	critical	Checkout submit handler redirecting captured data
<code>Obfuscated_Skimmer_Exfil_Beacon</code>	critical	Obfuscated JS beaconing captured form data
<code>Fake_Payment_Gateway_Iframe</code>	high	Injected iframe mimicking a payment gateway

phishing_social_engineering — lures

Rule	Severity	Catches
<code>Credential_Harvesting_Form</code>	high	Login form posting credentials to a non-relative target
<code>Brand_Impersonation_Lookalike_Markup</code>	medium	Markup referencing a known brand while hosted off-brand
<code>Urgency_Lure_With_Redirect</code>	medium	Urgency language paired with a redirect
<code>QR_Code_Phishing_Lure</code>	medium	Instructions to scan a QR code to resolve an account issue
<code>Invoice_Finance_Themed_Lure</code>	medium	Finance-themed lure pressuring a quick payment

THE PHISHING TAG GAP

These five rules carry the tag `phishing`, which is **not** in the engine's `SEVERITY_BY_TAG` map. That costs nothing here because every rule declares `severity` explicitly, so the tag fallback is never consulted.

It is worth knowing anyway: a future `phishing`-tagged rule that omits `severity` would silently default to medium rather than to anything the tag implies. Either add the tag to the map or keep declaring severity on every rule — the ruleset currently does the latter, on all 55.

linux_threats — Linux and container hosts

Rule	Severity	Catches
Linux_Reverse_Shell_OneLiner	critical	A transport primitive (/dev/tcp, nc -e) plus an interactive shell
LD_PRELOAD_Userland_Rootkit	critical	ld.so.preload alongside two or more libc hook symbols
SSH_Credential_And_Key_Theft	critical	Two SSH/history artifacts plus an archive or copy primitive
Container_Escape_Attempt	critical	Docker socket, release_agent, or nsenter with an action
Cron_Persistence_With_Download	high	A cron entry that fetches remote content and pipes it to a shell
Systemd_Service_Persistence	high	A unit launching from /tmp, /dev/shm, or a download
Linux_Log_And_History_Wiping	high	History disabled, or a log path paired with a wipe primitive

supply_chain — packages and build pipelines

Rule	Severity	Catches
NPM_Install_Hook_Executes_Code	critical	A manifest running code from pre/post-install
Python_Setup_Executes_On_Install	critical	setup.py cmdclass override running commands
CI_Secret_Harvesting	critical	Two CI secret names plus an outbound call
Package_Obfuscated_Payload	high	A package file decoding then executing a payload
Build_Pipeline_Tampering	high	A workflow fetching remote content and piping to a shell
Typosquat_Install_Beacon	high	Install-time host fingerprinting sent outbound

rmm_tunnel_abuse — remote access as intrusion tooling

These products are not malware, and their presence alone is not a finding. Every rule requires the automation context that separates an administrator from an intruder — no rule fires on a tool name by itself.

Rule	Severity	Catches
RMM_Unattended_Access_Password	critical	A remote access tool given a preset password
RMM_Silent_Unattended_Install	high	An RMM product installed with silent/quiet flags
Outbound_Tunnel_Tool_Configured	high	ngrok, cloudflared, frp or chisel configured to expose a port
RDP_Exposure_Tampering	high	fDenyTSConnections cleared plus a firewall change
Remote_Access_Service_Persistence	high	Remote access tooling wired to start at boot

cryptominer — resource abuse

Rule	Severity	Catches
XMRig_Stratum_Config_Strings	high	XMRig-style miner config or stratum pool connection
Mining_Process_Commandline_Flags	high	Command lines built with typical cryptomining flags
Browser-Based_Mining_Script	medium	A page embedding a browser cryptomining script

webshell — server backdoors

Rule	Severity	Catches
PHP_Webshell_Eval_Input	critical	PHP passing request data to an evaluation primitive
PHP_Webshell_Command_Exec	critical	PHP invoking a shell with attacker-controlled input
ASPX_Webshell	critical	ASP.NET page compiling or executing request-supplied code
JSP_Webshell	critical	JSP running an OS command from a request parameter

windows_threats — host tradecraft

Rule	Severity	Catches
Credential_Dumper_Indicators	critical	Strings from LSASS credential dumping tooling
Defense_Evasion_Commands	critical	Shadow copy deletion or security tooling disabled
Ransom_Note_Template	critical	Text reading like a ransom note beside encrypted files
PowerShell_Encoded_Command	high	PowerShell launched with a base64 encoded command
PowerShell_Download_Cradle	high	Download method paired with an execution method
Suspicious_Double_Extension	medium	Filenames masquerading with a double extension

demo — pipeline verification

Rule	Severity	Catches
High_Entropy_Executable	medium	A PE body above 7.2 bits per byte; requires the <code>math</code> module
EICAR_Test_File	low	The standard EICAR test string, harmless by design
Demo_TestKeyword	info	A literal test keyword; smoke test for the whole pipeline

HTTP API Reference

All endpoints are loopback-only and reject requests whose `Host` header is not a loopback name. Responses are JSON unless noted. No endpoint ever returns a credential.

Detection and triage

Method	Endpoint	Purpose
GET	<code>/api/state</code>	Stats, monitor status, engine info, telemetry, and effective configuration in one call.
GET	<code>/api/findings</code>	Filtered findings. Parameters: <code>severity</code> , <code>status</code> , <code>type</code> , <code>q</code> , <code>limit</code> .
GET	<code>/api/rules</code>	Loaded rules with namespaces and tags, plus compile errors by filename.
GET	<code>/api/telemetry</code>	Recent authentication failures. <code>force=1</code> bypasses the cache.
GET	<code>/api/stream</code>	Server-Sent Events: <code>hello</code> on connect, <code>event</code> per finding, <code>stats</code> every two seconds.
POST	<code>/api/scan</code>	<code>{"path": "..."} — scan a file or directory tree on demand.</code>
POST	<code>/api/monitor</code>	<code>{"action": "pause" "resume"}</code> .
POST	<code>/api/rules/reload</code>	Recompile the ruleset without restarting.
POST	<code>/api/triage</code>	<code>{"id", "status", "note"}</code> — set a finding's triage state.

Vulnerability intelligence

Method	Endpoint	Purpose
GET	/api/intel	Source lattice health. Reports whether the VirusTotal key is <i>configured</i> and <i>working</i> , never its value.
GET	/api/nvd	NVD cache status: records cached, sync window, rate limit, TLS bundle in use, last error.
GET	/api/nvd/cves	Cached CVEs, highest CVSS first. Parameters: <code>limit</code> , <code>severity</code> .
GET	/api/nvd/cve	<code>?id=CVE-2021-44228</code> — one record, cached on disk after first fetch.
GET	/api/nvd/search	<code>?q=log4j&limit=20</code> — live keyword search against the NVD API.
POST	/api/nvd/sync	<code>{"days": 3}</code> — sync the trailing window into the local cache.
GET	/api/osv	OSV adapter status: cached queries, queries this session, last error.
GET	/api/osv/query	<code>?commit=<sha></code> , <code>?purl=<purl></code> , or <code>?package=&ecosystem=&version=.</code>
POST	/api/osv/query	The same, as a JSON body: <code>{"commit": "..."}.</code>
GET	/api/vt	VirusTotal adapter status and the tier actually detected.
GET	/api/vt/capabilities	What the key may reach, probed rather than assumed.
GET	/api/vt/file	<code>?hash=</code> — engine verdicts, family label, first-seen, permalink.
GET	/api/vt/livehunt	Enterprise-gated. Returns 402 with the reason on a public key.

Extracted indicators

Method	Endpoint	Purpose
GET	/api/iocs	Indicators aggregated across findings, ranked by how many distinct files they appear in.
GET	/api/iocs?type=	Filter to one observable type (url, domain, ipv4, btc, cve, ...).
GET	/api/iocs?scope=external	Drop private, loopback and reserved addresses.
GET	/api/iocs?format=csv	CSV export carrying both raw and defanged values.

Detection event shape

```
{
  "id": "a3f19c2b8e04",
  "timestamp": "2026-09-11T14:40:47-05:00",
  "event_type": "yara_match",
  "severity": "critical",
  "verdict": "malicious",
  "trigger": "monitor",
  "file_path": "C:\\...\\uploads\\README_RESTORE.txt",
  "file_size": 97,
  "sha256": "2f20176c2252fac5e23e2f6e4221708af75db0b8407beb24c9fa1ec1b21e0435",
  "entropy": 4.4,
  "scan_ms": 1.21,
  "rule_names": ["Ransom_Note_Template"],
  "matches": [{
    "rule": "Ransom_Note_Template",
    "namespace": "windows_threats",
    "severity": "critical",
    "strings": [
      {"identifier": "$a1", "offset": 0, "preview": "Your files have been encrypted"},
      {"identifier": "$b1", "offset": 40, "preview": "bitcoin"}
    ]
  }],
  "telemetry": {"status": "denied", "source": "windows_security_log",
    "window_minutes": 5, "count": 0},
  "status": "new"
}
```

CVE record shape

```
{
  "id": "CVE-2021-44228",
  "published": "2021-12-10T10:15:09.143",
  "last_modified": "2025-04-03T14:15:19.153",
  "status": "Modified",
  "score": 10.0,
  "cvss_severity": "CRITICAL",
  "severity": "critical",
  "cvss_version": "3.1",
  "vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H",
  "cwe": ["CWE-20", "CWE-400", "CWE-502", "CWE-917"],
  "description": "Apache Log4j2 2.0-beta9 through 2.15.0 ... JNDI features ...",
  "references": 103,
  "source": "security@apache.org",
  "cached": true
}
```

Configuration Reference

Defaults live in `securitysuite/config.py`. To override, create `config.json` beside `run.py` with only the keys you want to change; unknown keys are reported and ignored. **Credentials are never read from or written to `config.json`** — they come from the environment or `.env`.

config.json

Key	Default	Meaning
<code>watch_paths</code>	<code>["uploads"]</code>	Directories to monitor. Created if absent.
<code>recursive</code>	<code>true</code>	Include subdirectories.
<code>poll_interval</code>	<code>2.0</code>	Seconds between directory sweeps.
<code>settle_seconds</code>	<code>1.0</code>	A file must stop changing this long before it is scanned.
<code>max_file_mb</code>	<code>64</code>	Files above this are recorded as skipped, not scanned.
<code>ignore_suffixes</code>	<code>.tmp .part .crdownload .swp</code>	Extensions never scanned.
<code>scan_existing_on_start</code>	<code>false</code>	Whether to scan files already present at startup.
<code>rules_dir</code>	<code>rules</code>	Searched recursively for <code>.yar</code> and <code>.yara</code> .
<code>lookback_minutes</code>	<code>5</code>	Telemetry correlation window.
<code>auth_log_path</code>	<code>/var/log/auth.log</code>	First text auth log tried on POSIX. <code>/var/log/secure</code> and <code>/var/log/authlog</code> are tried next, then journald. Ignored on Windows.
<code>telemetry_cache_seconds</code>	<code>15.0</code>	How long a telemetry result is reused.
<code>max_telemetry_events</code>	<code>25</code>	Cap on events attached to one finding.
<code>findings_log</code>	<code>data/findings.ndjson</code>	Append-only alert log. Keep it outside every watched path.
<code>triage_file</code>	<code>data/triage.json</code>	Mutable triage state, separate from the log.
<code>history_limit</code>	<code>2000</code>	Ring buffer size; also lines replayed at startup.
<code>nvd_cache_dir</code>	<code>nvds</code>	Where the CVE cache and per-CVE lookups are written.
<code>nvd_sync_days</code>	<code>3</code>	Default trailing window for a sync. NVD caps ranges at 120 days.
<code>nvd_max_records</code>	<code>4000</code>	Upper bound on records pulled in one sync.
<code>osv_cache_dir</code>	<code>data/osv</code>	Where OSV answers are cached, keyed by a hash of the query.
<code>vt_cache_dir</code>	<code>data/vt</code>	Where VirusTotal hash reports are cached.
<code>host / port</code>	<code>127.0.0.1 / 8787</code>	Dashboard bind address. Do not bind a routable interface.

.env

Gitignored. `.env.example` is committed with empty values to document what exists. An existing environment variable always takes precedence over the file.

Variable	Meaning
<code>VIRUSTOTAL_API_KEY</code>	Required for the VirusTotal adapter — no key, no data. Absent means the card reads <code>optional</code> and nothing is requested.
<code>NVD_API_KEY</code>	Optional. NVD works without one at 5 requests per 30 s; a free key raises that to 50 per 30 s and drops the client's minimum interval from 6.5 s to 0.8 s.

Command Line and Recipes

Invocation

```
python run.py                                # monitor + dashboard, opens a browser
python run.py --watch D:\ftp --watch E:\inbox
python run.py --port 9000 --no-browser
python run.py --scan C:\Users\me\Downloads    # one-shot scan, JSON to stdout, exit
python run.py --headless                     # monitor only, no web server
python run.py --scan-existing                # also scan files already present at startup
python run.py --rules D:\team-rules          # alternate rules directory
```

On Windows a desktop shortcut targeting `python.exe run.py`, with the project as its working directory, launches the whole console in one click.

Desktop launcher

```
python install_shortcut.py                  # create a launcher for this OS
python install_shortcut.py --remove        # remove it
```

Platform	Creates	Notes
Windows	Security Suite.lnk	Uses <code>assets/securitysuite.ico</code> .
macOS	Security Suite.command	Double-clickable; macOS may confirm on first open.
Linux	security-suite.desktop	Written to both <code>~/.local/share/applications</code> and the desktop. Some desktops need <i>Allow Launching</i> first.

Verifying an installation

The sample files each trip exactly one rule, which makes them a fast end-to-end check:

```
copy samples\README_RESTORE.txt uploads\
```

Sample	Rule	Severity
README_RESTORE.txt	Ransom_Note_Template	critical
cleanup_commands.txt	Defense_Evasion_Commands	critical
dump_notes.txt	Credential_Dumper_Indicators	critical
task_setup.log	PowerShell_Encoded_Command	high
update_helper.txt	PowerShell_Download_Cradle	high
mail_body.txt	Suspicious_Double_Extension	medium
report_q3.txt	(nothing — clean scan)	—

WHY THE SAMPLES ARE PLAIN TEXT

A genuine .php web shell or PE sample written to disk is quarantined by antivirus before the scanner can read it — the failure described in Chapter 6. Text samples survive, so the smoke test is reliable on a protected endpoint. The web shell rules are verified in memory instead.

Intel checks

```
# Is the NVD cache warm, and which CA bundle is in use?
curl -s http://127.0.0.1:8787/api/nvd | jq '{cached, last_sync, rate_limit, tls_bundle}'

# Sync the trailing week
curl -s -X POST http://127.0.0.1:8787/api/nvd/sync \
  -H "Content-Type: application/json" -d '{"days": 7}' | jq

# Look up one CVE
curl -s "http://127.0.0.1:8787/api/nvd/cve?id=CVE-2021-44228" | jq '{id, score, severity, cwe}'

# Is a source commit affected by anything known?
curl -s -X POST http://127.0.0.1:8787/api/osv/query \
  -H "Content-Type: application/json" \
  -d '{"commit": "6879efc2c1596d11a6a6ad296f80063b558d5e0f"}' | jq '.vulns[]'

# Which adapters are actually live?
curl -s http://127.0.0.1:8787/api/intel | jq '.sources[] | {id, status}'
```

Querying the findings log

```
# Critical findings only
jq -c 'select(.severity=="critical")' data/findings.ndjson

# Unique hashes for an estate-wide search
jq -r '.sha256' data/findings.ndjson | sort -u

# Rule hit counts, most frequent first
jq -r '.rule_names[]' data/findings.ndjson | sort | uniq -c | sort -rn

# Findings that had correlated authentication failures
jq -c 'select(.correlated_auth_failures > 0)' data/findings.ndjson
```

Further reading

- **YARA documentation** — yara.readthedocs.io. The module reference, particularly `pe` and `math`, repays a careful read.
- **NVD API documentation** — nvd.nist.gov/developers/vulnerabilities. Pagination, date-window limits, and rate limits are all specified there.
- **OSV API** — google.github.io/osv.dev/api/. Query by commit, package, or purl; also documents `querybatch` for bulk lookups.
- **CISA KEV** — cisa.gov/known-exploited-vulnerabilities-catalog. The subset of CVEs actually being exploited; the highest-value filter in vulnerability work.
- **MITRE ATT&CK** — attack.mitre.org. Useful for mapping rules to technique IDs, which makes coverage gaps visible.
- **The Sigma project** — the same reasoning applied to log events rather than files. Chapter 4's capability-plus-context pattern transfers directly.
- **The repository** — github.com/gocar112/security-suite-dashboard. Every listing in this book is taken from it.